

TECHNICAL STUDY GUIDE

The ER DSL Compiler Architecture

Salesforce ER Modeller Studio



Recognize

→

Normalize

→

Model

→

Lay out

→

Emit

WRITTEN BY

Vikas Cohen

Compiler Architect and System Designer

Contents

32 parts · 23 diagrams

•	Abstract	3
•	Reading guide and theory map	3
1	The philosophy behind the compiler	5
2	Why the design is reusable	9
3	The compiler pipeline	12
4	Main implementation modules	15
5	What a compiler does in this domain	18
6	Lexical analysis: scanning the text	21
7	Indentation and tree construction	24
8	Grammar and top-level dispatch	27
9	Recursive descent parsing for conditions	31
10	Name resolution and symbol tables	34
11	Stateful interpretation and safe mutation	37
12	Intermediate representation and step generation	40
13	Semantic validation and domain-aware checks	44
14	User conditions vs. record criteria: two expression languages	47
15	Applying steps and graph expansion	50
16	Access calculation and analysis	53
17	Optimization and performance thinking	56
18	Diagnostics and error recovery	59
19	The code-generation model in the DSL	62
20	Editor integration and language services	65
21	Testing the compiler properly	68
22	Complete example end-to-end	71
23	Extending the language	74
24	Current limitations and future improvements	77
25	Final architecture summary	80
26	Design principles every compiler author can reuse	83
27	Closing thought	85
•	Appendix A: A complete grammar and test derivation	88
•	Appendix B: Glossary of terms	91
•	Appendix C: Further reading	94

Abstract

This document presents an implementation-oriented and academically structured account of the Entity-Relationship (ER) domain-specific language (DSL) compiler used by Salesforce ER Modeller Studio. The compiler translates line-oriented declarations into a normalized semantic graph and then into geometry and target-specific artifacts. Its design illustrates core compiler principles—recognition, semantic analysis, intermediate representations, diagnostics, code generation, and testing—without introducing machinery that the language does not require.

The semantic model is authoritative. Geometry, SVG, Mermaid, draw.io XML, and PNG are derived representations. This invariant allows all renderers to share one interpretation of the source.

Reading guide and theory map

◆ How this expanded edition is organised

The original chapters are kept exactly as you wrote them. After each one there is a **Going deeper** section that explains the computer science and the logic underneath the design decision.

This compiler is deliberately small. It has a flat, line-oriented, declarative language with no nesting, no precedence, no expressions and no execution. In a conventional compiler guide, chapters 7 (trees), 9 (recursive descent), 14 (expression languages), 15 (executing steps) and 16 (access calculation) describe large pieces of machinery. Here they describe the **negative space**: machinery that theory says you can leave out, and the precise change that would bring it back. Understanding *why* a stage is absent is as valuable as understanding a stage that is present, because it is what lets you choose the right amount of machinery for the next language you build.

Where a topic is not conventional, the deeper sections change focus. They lean on regular languages and regular-expression engines, on graph theory and graph drawing, on canonical data models, on output encoding, and on invariant-based testing, rather than on parser generators and interpreters.

THEORY

Background from computer science, explained from first principles: automata, graphs, encodings, complexity.

FORMAL VIEW

The same idea stated precisely with sets, functions and equations. You can skim these on a first read.

WORKED EXAMPLE

A concrete trace or calculation that ties the theory back to the ER DSL. Where a detail is not stated in the original chapters, the box says so.

COMMON PITFALL

A mistake that is easy to make in a compiler of this kind, and how to avoid it.

CHECK YOUR UNDERSTANDING

Open-ended questions for self-study or team discussion.

◆ The theory map

CHAPTERS	COMPILER STAGE	THEORY YOU WILL MEET
1	Philosophy	declarative semantics, the proportionality principle, canonical data models
2, 4	Separation of concerns	design by contract, functional core with an imperative shell, cohesion and change axes
3, 19	Pipeline and generation	derivation graphs, memoisation, output encoding, injectivity, determinism
5, 15	The domain as a graph	the entity-relationship model, labelled multigraphs, reachability, topological order
6, 8	Recognition	regular languages, finite automata, backtracking engines, ordered choice
7, 9	Structure that is absent	the Chomsky hierarchy, pumping intuition, deforestation, choosing a parsing strategy
10	Names	equivalence relations, interning, forward references, implicit declaration
11, 12	State and representation	folds, semilattices, idempotence, IR levels, graph drawing
13, 14	Validation and conditions	open and closed worlds, invariants, views as queries, three-valued logic
16	Policy versus structure	typed graphs, deny-by-default, closed-world reasoning
17	Performance	complexity by stage, memoisation, level of detail, Amdahl's law
18	Diagnostics	error recovery in line-oriented languages, structured diagnostics
20	Language services	prefix matching, single source of truth, round trips
21	Testing	properties, metamorphic relations, input-space partitioning

CHAPTERS	COMPILER STAGE	THEORY YOU WILL MEET
22 – 27	The whole system	conservative extension, fitness functions, decision records

◆ Notation used in the formal boxes

SYMBOL	READ AS	SYMBOL	READ AS
$G = (V, E)$	a graph with vertices V and edges E	$\forall \exists$	for all, there exists
$\in \subseteq$	is a member of, is a subset of	$\wedge \vee \neg$	and, or, not
$\cup \cap \setminus$	union, intersection, difference	$\Rightarrow$	implies
$\emptyset$	the empty set	$\llbracket P \rrbracket$	the meaning of program P
$\sim$	is equivalent to	$\sqcup$	join (least upper bound)
$\mapsto$	maps to	$\circ$	function composition

1

The philosophy behind the compiler

The language is declarative rather than operational. A program specifies entities, fields, and relationships; it does not specify an execution sequence. Its meaning is therefore structural: the compiler constructs a labeled graph and translates that graph into visual targets.

A useful abstraction is:

```
source text -> semantic model -> optional geometry IR -> target artifact
```

`parseEr()` combines line classification, syntactic recognition, and early semantic normalization because the grammar is flat. This is a deliberate application of proportionality. A token stream, parser generator, or AST would become valuable if the language acquired nesting, precedence, blocks, or multiline constructs; for the current grammar, they would add complexity without improving correctness.

◆ Going deeper: declarative programs and their meaning

The opening chapter says the language is *declarative rather than operational*. Computer science makes this precise. An **imperative** program describes a computation: the order of the statements matters, and the meaning is what the machine does. A **declarative** program describes a *result*: it states facts, and the meaning is the structure those facts define.

	IMPERATIVE LANGUAGE	DECLARATIVE LANGUAGE (THIS DSL)
A program is	a sequence of instructions	a set of statements about the world
Does order matter?	yes, the state changes	in principle no, statements do not depend on one another
Meaning is defined by	execution (operational semantics)	the structure described (denotational semantics)
The compiler must	build a machine to run it	build a data structure that represents it
Example	the SML scripts in the companion guide	an ER diagram description

Because nothing runs, this compiler needs no *operational* semantics at all. Its whole meaning fits in one sentence: **a program denotes a labelled graph**.

FORMAL VIEW The meaning of a program

Let a model be a graph $G = (V, E)$ whose vertices are entities carrying their fields, and whose labelled edges are relationships. Each line of a program contributes a small piece of graph, and the meaning of the whole program is the union of the pieces:

$$\llbracket P \rrbracket = \sqcup \{ \llbracket \ell \rrbracket : \ell \text{ is a line of } P \}$$

An entity line contributes one vertex and some fields. A relationship line contributes two vertices (the child and the parent, if not yet seen), one relationship field, and one edge. Because the operator $\sqcup$ is a union-like join, **the order of the lines does not change the graph**, only how it is presented (chapter 11 examines exactly what "presented" means).

◆ The proportionality principle

The chapter states that a token stream, a parser generator or an AST would add complexity without improving correctness for this grammar. This is an application of a general principle: **match the machinery to the structure of the language**. Formal-language theory supplies the yardstick. Each kind of language feature demands a specific amount of computational power, and features that do not appear should not be paid for.

LANGUAGE FEATURE	MACHINERY IT DEMANDS	LANGUAGE CLASS	IN THE ER DSL
Lines with a fixed shape	one regular expression per line kind	regular	present
A leading keyword that selects the line kind	dispatch on the first word	regular	present
Names that refer to earlier or later declarations	a symbol table	semantic, beyond any grammar	present
Nested blocks or parentheses	a stack: recursive descent and an AST	context-free	absent
Operator precedence	a stratified grammar or precedence climbing	context-free	absent
Declarations that span several lines	a scanner with state, or line joining	regular with state	absent
Control flow and execution order	an interpreter or an IR with control flow	Turing complete	absent

Reading down the last column shows a compiler built to exactly its grammar. Every "absent" row is a real cost avoided: code to write, tests to maintain, and bugs to fix. The principle cuts both ways, and the chapter states the other half clearly: when a row flips from absent to present, the machinery in the middle column must arrive with it.

◆ A canonical model at the centre

Chapter 1 declares the semantic model *authoritative* and every artefact *derived*. In enterprise integration this is a well-known pattern, the **canonical data model** (Hohpe and Woolf, 2003): instead of translating every format directly into every other, translate each one into and out of a single agreed representation. The arithmetic is the same as the $M \times N$ argument for intermediate representations.

SOURCES	TARGETS	DIRECT TRANSLATORS	THROUGH A CANONICAL MODEL
1 (DSL text)	4 (canvas, Mermaid, draw.io, PNG)	4 interpretations of the text	1 parser + 4 emitters
2 (DSL text and imported metadata)	4	8	$2 + 4 = 6$
2	5 (a new target)	10	$2 + 5 = 7$

The last two rows show the payoff: adding a fifth target costs one new emitter and does not reopen the question of what the source means.

KEY INSIGHT

"The semantic model is authoritative" is an invariant, and invariants are what allow independent parts of a system to change safely. As long as every target is a function of the model, targets cannot disagree about what the source said. They can only disagree about how to draw it.

COMMON PITFALL Renderer-first extension

The commonest way to break the invariant is to add a feature in a renderer because it is quick, for example special-casing an entity name when drawing. The feature now exists in one target only, the model does not know about it, and the other targets silently disagree. Add the concept to the model first (chapter 23).

CHECK YOUR UNDERSTANDING

1. Explain in your own words why the order of declarations should not change the meaning of a program in this DSL, and identify one thing that *can* change.
2. Which rows of the machinery table would flip first if the DSL gained namespaces, and what would you add?
3. Suppose a new target, say a SQL schema generator, is proposed. List the questions you would ask of the semantic model before writing it.

2

Why the design is reusable

The architecture separates six concerns: source acquisition, syntactic recognition, semantic normalization, domain-model construction, geometry generation, and target rendering. Each boundary has a contract. The parser returns a model or diagnostics; the geometry builder returns coordinates and paths; exporters return target artifacts.

This separation is reusable because meaning is not mixed with presentation. A relationship's kind is semantic; its connector lane is geometric; Mermaid identifier sanitization is target-specific. The semantic model can consequently support the live canvas, Mermaid, draw.io, and PNG without duplicating interpretation logic.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: contracts at every boundary

The chapter says each boundary has a contract. Bertrand Meyer's **design by contract** gives that idea a vocabulary. For each function, the *precondition* states what the caller must guarantee, the *postcondition* states what the function guarantees in return, and an *invariant* states what stays true of the data throughout.

BOUNDARY	PRECONDITION	POSTCONDITION AND INVARIANTS
<code>parseEr(text)</code>	the input is a string	returns a model (or diagnostics); every identity appears once; no exact duplicate relationships; fields carry stable defaults
<code>buildErGeometry(model, overrides)</code>	the model satisfies the parser's invariants	every box has positive width and height; every connector has a path; canvas bounds contain all boxes
Exporter	a model or geometry that satisfies the above	a string that is well-formed in the target grammar; the same input gives the same output

Contracts are the reason each stage can be tested alone (chapter 21): a test for geometry can *assume* the model invariants and needs no parser.

◆ Three kinds of information

The chapter's own example is a good test: a relationship's kind is *semantic*, its connector lane is *geometric*, and the Mermaid identifier sanitisation is *target-specific*. Every piece of data in the system belongs to exactly one of these three kinds, and putting it in the wrong place is the root of most maintenance pain.

KIND	QUESTION IT ANSWERS	BELONGS IN	EXAMPLES
Semantic	What does the source say?	the model	entity identity, field markers, relationship kind
Presentational	How should it look on the canvas?	geometry and saved overrides	box positions, connector lanes, hidden-field counts
Target-specific	How does <i>this output format</i> spell it?	one exporter	Mermaid identifier rules, XML escaping, draw.io cell ids

A quick way to classify a new piece of data is to ask two questions. *Would it change if the source text changed?* If yes it is probably semantic. *Would it change if I switched to a different renderer?* If yes it is presentational or target-specific.

WORKED EXAMPLE Where does the hidden-field count live?

When an entity has more fields than can be shown, the box displays a "+N more" row. The **list of fields** is semantic and lives in the model. The **decision to hide some** depends on box height and screen space, so it is presentational and lives in geometry. If the count were stored in the model, changing the font size would appear to change the meaning of the program.

◆ Saved positions are keyed by identity

The chapter notes that saved coordinates are presentation state, not source semantics. There is a subtle consequence for the identity rules of chapter 10. If positions are saved under the entity's *display name*, then editing `account` to `Account` may cause the saved position to be lost, although the entity is the same. Keying presentation state by the **normalised identity** (the lower-case key) makes it survive cosmetic edits.

◆ Derivations should be pure

If geometry is a pure function of the model and the overrides, and each exporter is a pure function of its input, then the whole system becomes easy to reason about and to cache. Purity, as introduced in the earlier guide, means the output depends only on the inputs and nothing else is changed. It is the property that turns the boundaries above into genuine contracts.

COMMON PITFALL Presentation leaking into the model

A tempting shortcut is to store an x and y coordinate, or a colour, directly on a field or entity in the model. It works until a second target needs different coordinates, or until two users open the same source with different layouts. Keep the model free of anything that a different renderer would want to change.

CHECK YOUR UNDERSTANDING

1. Classify each of these as semantic, presentational or target-specific: the entity name, the number of visible rows, the escaped form of a label in XML, a self-loop's radius, the field type text.
2. State the postcondition of `parseEr` for the input `Contact.AccountId -> Account` in terms of the invariants.
3. What could go wrong if saved layouts were keyed by array index instead of identity?

3

The compiler pipeline

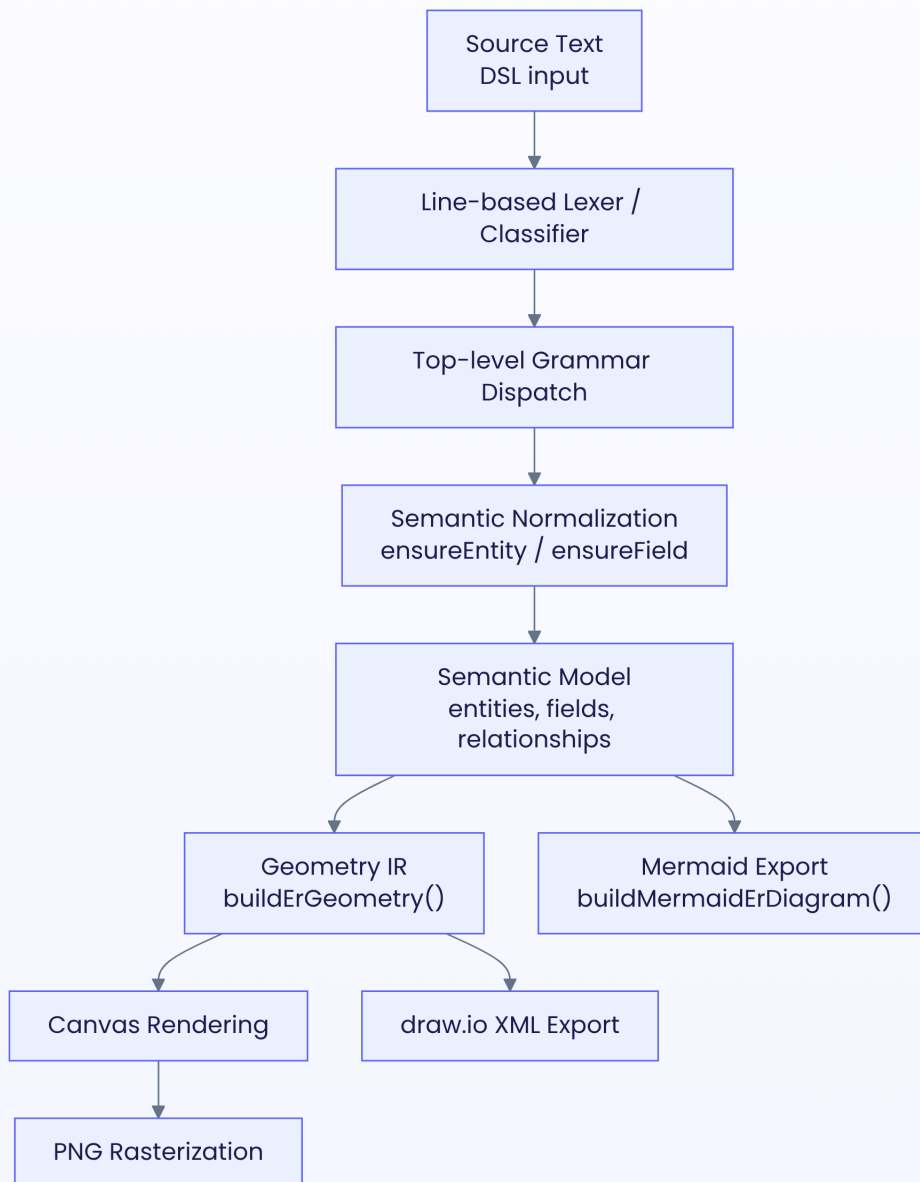


FIGURE 1 The compiler pipeline

The stages are: split input into lines; classify each line; extract components; intern names and fields; normalize relationships; optionally compute geometry; and emit target syntax. Quoted Mermaid labels are intentional because braces, parentheses, slashes, and HTML breaks can otherwise be parsed as diagram syntax.

◆ Going deeper: a pipeline that is really a graph

The diagram in this chapter looks like a pipeline but is not a chain. After the semantic model, the flow **fans out**: one branch goes to Mermaid, another goes through geometry to the canvas and to draw.io, and the canvas continues to PNG. Mathematically it is a *directed acyclic graph* of derivations.

FORMAL VIEW The derivation graph as functions

Writing each arrow as a function makes the structure exact:

```

model    = parse (source)
geometry = layout (model, overrides)
mermaid  = emitMermaid (model)
drawio   = emitDrawio (geometry)
svg      = render (geometry)
png      = rasterise (svg)

```

Every artefact is a function of earlier ones, and nothing depends on a later stage. A valid evaluation order is any **topological order** of the graph, for instance: parse, then layout and emitMermaid, then emitDrawio and render, then rasterise.

The functional view immediately answers a practical question: *what must be recomputed when something changes?* Only the artefacts downstream of the change.

WHAT CHANGED	RECOMPUTED	NOT RECOMPUTED
The source text	everything	nothing
A dragged box position (an override)	geometry, canvas, draw.io, PNG	the model, Mermaid
Export options for draw.io	draw.io	model, geometry, everything else
Mermaid output style	Mermaid	everything else

This is **memoisation** on a derivation graph: keep each result together with the inputs that produced it, and reuse it while the inputs are unchanged. It is safe because every step is pure.

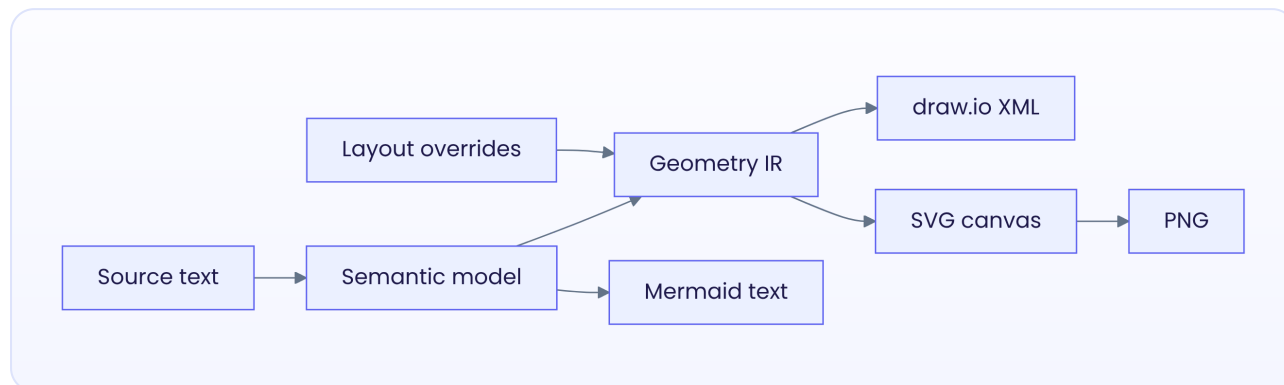


FIGURE 2 Going deeper: a pipeline that is really a graph

◆ Quoting is output encoding

The chapter mentions that Mermaid labels are quoted because braces, parentheses, slashes and HTML breaks can otherwise be parsed as diagram syntax. This is the same problem that produces SQL injection and cross-site scripting, and it has the same solution: **encode data for the context it is being placed into**. Characters that are harmless in one grammar are metacharacters in another.

FORMAL VIEW Encoding correctness

For any target grammar, an *encoder* is correct when reading the encoded text back gives the original string:

$$\text{decode}(\text{parse_target}(\text{encode}(s))) = s \quad \text{for every string } s$$

A quoted Mermaid label satisfies this for text that contains parentheses. An unquoted label does not, because `(` changes how the line is parsed. A test that feeds hostile strings through every encoder and checks this equation catches whole classes of bugs.

OUTPUT CONTEXT	SPECIAL CHARACTERS TO HANDLE	WHERE THE ENCODER LIVES
Mermaid label or identifier	quotes, braces, parentheses, slashes, line breaks	the Mermaid exporter
draw.io XML attribute	& < > " and control characters	the draw.io exporter
SVG text	& < >	the canvas renderer
Canvas 2D text	none: text is drawn directly, not parsed	nowhere

COMMON PITFALL Escaping in the parser

It is tempting to sanitise names once, when parsing, so that "every later stage is safe". The problem is that the right escaping depends on the destination, and the model must keep the original text (a semantic fact). Escape at each *output boundary*, and only there.

CHECK YOUR UNDERSTANDING

1. Draw the dependency graph again with "layout overrides" as an input. Which artefacts depend on it?
2. Why is PNG generation not independent of the model, even though the model is not its direct input?
3. Give one string that is safe as a canvas label but breaks an unquoted Mermaid label.

4**Main implementation modules**◆ **4.1** `erDiagramLogic.js`

This is the compiler core. It owns parsing, semantic construction, geometry generation, connector routing, Mermaid generation, draw.io generation, and domain helpers. Keeping these operations near the model makes them independently testable.

◆ **4.2** `diagramStudio.js`

This is the application host. It owns editor state, persisted positions, resizing, interaction, and compiler invocation. It supplies source text to `parseEr()` and layout overrides to `buildErGeometry()`.

◆ **4.3** `buildErGeometry()`

This creates the visual IR: entity boxes, dimensions, visible rows, hidden-field counts, connector paths, self-loops, lane offsets, and canvas bounds. Saved coordinates are presentation state, not source semantics.

◆ **4.4** Export generators

Mermaid consumes the semantic model and performs layout itself. Canvas and draw.io consume geometry. PNG rasterizes generated SVG through a separate utility. All targets derive from the same semantic interpretation.

◆ Going deeper: one core, several change axes

`erDiagramLogic.js` owns parsing, semantic construction, geometry, connector routing, Mermaid, draw.io and helpers. The chapter defends this: keeping the operations near the model makes them independently testable. Modularity theory offers a way to think about when that stays true and when it stops.

Cohesion describes how strongly the contents of a module belong together, and the principle of **single responsibility** refines it: a module should have *one reason to change*. In this core module there are at least three independent reasons to change:

AXIS OF CHANGE	WHAT TRIGGERS IT	FUNCTIONS AFFECTED
Language	new syntax or markers	recognition and normalisation
Visual design	new box style, routing rules	geometry and connector routing
Target formats	a new Mermaid or draw.io feature	the exporters

A file with three change axes is not wrong. It is *acceptable* while the axes stay small and are edited by the same people, and it is a candidate for splitting when signals accumulate: the file becomes hard to navigate, unrelated changes keep colliding in review, or its tests cannot run without setup that belongs to another axis. A useful empirical signal is **change coupling**: if two groups of functions are never changed in the same commit, they probably belong in different modules.

◆ Functional core, imperative shell

The host module `diagramStudio.js` owns editor state, persisted positions, resizing and interaction. The core owns the pure translation. This is the pattern known as **functional core, imperative shell**: decisions live in pure functions that are trivial to test, and effects (the DOM, storage, events) live in a thin outer layer.

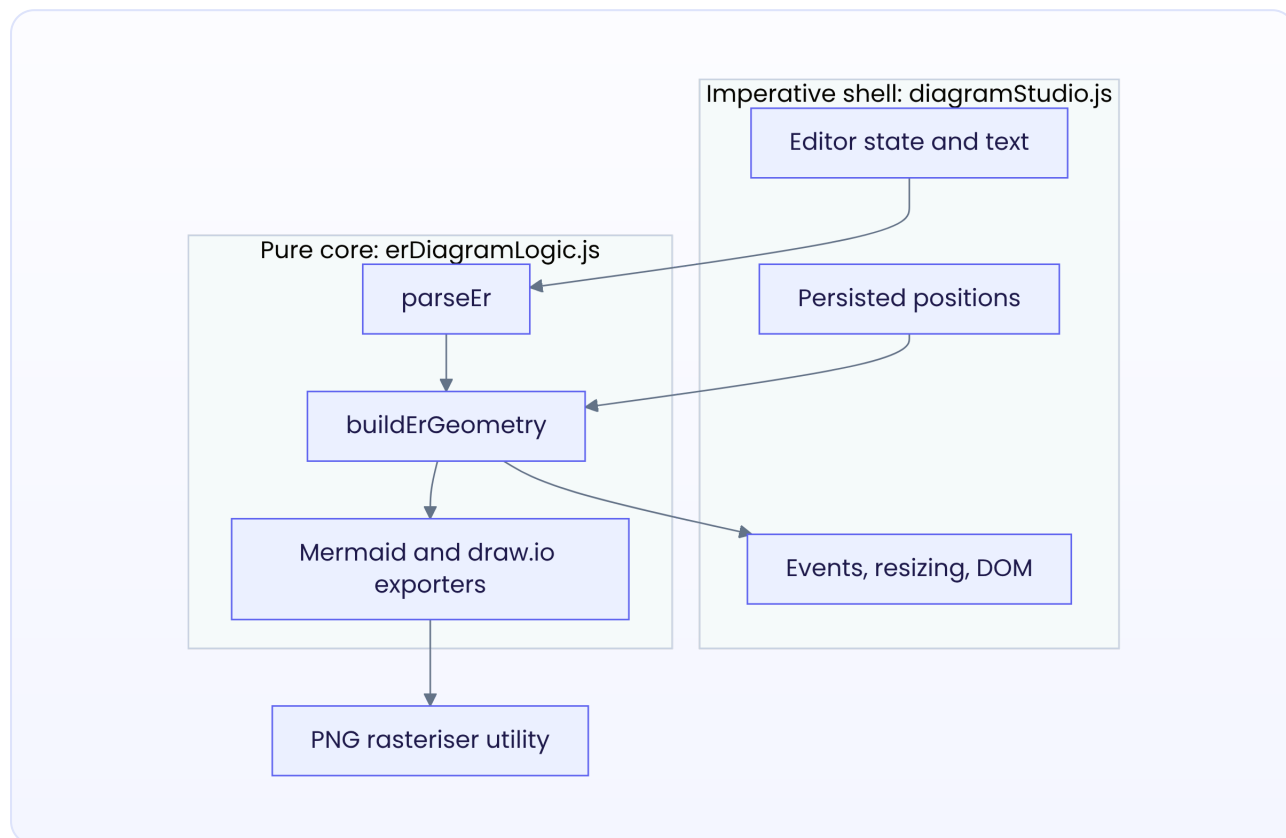


FIGURE 3 Functional core, imperative shell

The direction of dependency matters as much as the split. The shell may call the core, but the core must never reach back into the shell or the browser. That one rule is what allows `parseEr()` and `buildErGeometry()` to be tested in plain Node without a page, as chapter 12 observes.

FUNCTION	INPUTS	OUTPUT	PURE?
<code>parseEr(text)</code>	source text	semantic model	yes
<code>buildErGeometry(model, overrides)</code>	model, saved positions	visual IR	yes
Exporters	model or geometry	text in a target grammar	yes
Rasteriser	SVG text	image data	no: typically relies on browser image and canvas APIs
Shell handlers	user events	state changes, DOM updates	no

◆ Deep interfaces

John Ousterhout calls a module *deep* when it has a small interface that hides a lot of work.

`parseEr(text)` is deep: one string in, a fully normalised model out. Its opposite, a *shallow* module, forces callers to know many details. The state of the geometry function is a good design review question: does the shell need to know anything about connector routing, or does it only supply overrides and consume boxes and paths?

COMMON PITFALL The core that imports the shell

A convenience import of a UI helper into the core, for instance to measure text, makes the core untestable without a browser and breaks the layering silently. If the core needs text width, accept a measuring function as an argument (dependency injection) so tests can supply a simple stub.

CHECK YOUR UNDERSTANDING

1. List three signals that would convince you to split `erDiagramLogic.js`, and the module boundaries you would draw.
2. Which functions in the table must the tests stub, and which can they call directly?
3. What would change if geometry needed the width of rendered text?

5

What a compiler does in this domain

The compiler reads text, recognizes declarations, extracts names and annotations, resolves identities, normalizes duplicates, constructs a graph, diagnoses invalid input, and translates the graph to target languages. Entities are nodes, fields are node attributes, and relationships are directed edges.

The compiler does not execute business logic, query Salesforce, or calculate authorization. A feature belongs here when it changes accepted syntax, source meaning, or target meaning; user-interface behavior and pixel placement are downstream concerns.

◆ Going deeper: the entity-relationship model as a graph

The domain this compiler serves has an academic ancestor. Peter Chen's **entity-relationship model** (1976) describes data as *entities* (things), *attributes* (properties of things) and *relationships* (associations between things). It sits next to Codd's **relational model** (1970), in which the same ideas become tables, columns and foreign keys. The DSL is a text notation for this model, and the platform it targets has the same structure under different names.

ER CONCEPT	DSL CONSTRUCT	SALESFORCE CONCEPT	RELATIONAL CONCEPT	GRAPH CONCEPT
Entity	<code>entity Account</code>	an object	a table	a vertex
Attribute	<code>Name , AnnualRevenue</code>	a field	a column	a property of a vertex
Relationship	<code>Contact.AccountId - > Account</code>	a relationship field on the child	a foreign key	a directed edge from child to parent
Relationship kind	the operator <code>-></code> , <code>=></code> or <code>~></code>	the kind of relationship field	a constraint on the foreign key	a label on the edge

The chapter says entities are nodes, fields are node attributes and relationships are directed edges. The exact mathematical object is a **labelled directed multigraph**.

FORMAL VIEW

The model as a graph

$$G = (V, F, E) \quad V \text{ entities} \quad F : V \rightarrow \text{sets of fields} \quad E \subseteq V \times \text{Field} \times V \times \text{Kind}$$

An edge (*child*, *field*, *parent*, *kind*) points from the entity that holds the relationship field to the entity it refers to. The word **multigraph** matters: two entities may be connected by several edges (different fields, or different kinds), and an edge may connect an entity to itself. Both situations occur in real schemas, and the chapter on testing lists them explicitly (self-relationships and parallel edges).

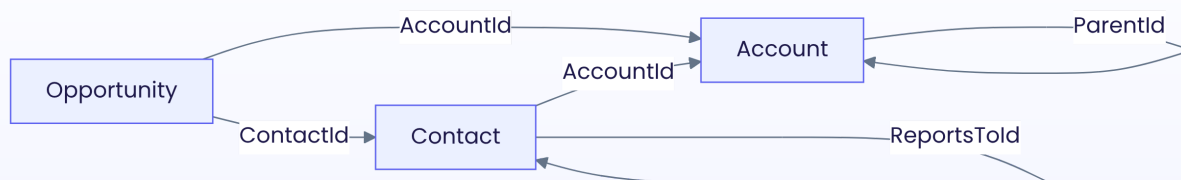


FIGURE 4 Going deeper: the entity-relationship model as a graph

The figure is a small schema. Account is a **hub**: two other entities point at it, so it has a high *in-degree*. Two entities point at themselves (a self-loop, typical for hierarchies such as parent accounts or reporting lines). Opportunity has two outgoing edges and so a higher *out-degree*. These structural

counts are the basis of the analyses discussed in chapter 15.

◆ What a relationship field means

A relationship field on the child holds at most one reference to a parent record. From the point of view of the *data*, each relationship edge is therefore a partial function from child records to parent records: many children may share one parent, and a child may have no parent if the field is optional. That is exactly the ER notion of a **many-to-one** relationship, and it explains why Mermaid draws the parent end as "exactly one" or "zero or one" and the child end as "zero or many".

◆ The boundary of the language

The chapter is emphatic about what the compiler does *not* do: it does not execute business logic, query Salesforce or calculate authorization. That is the boundary between **schema description** and everything that operates *on* schemas. It is worth stating the rule that decides where a feature belongs: a feature belongs in the compiler if it changes **accepted syntax, source meaning or target meaning**. If it only changes how a diagram looks or behaves, it belongs downstream.

QUESTION	IF YES, THE FEATURE BELONGS IN
Does it change which text is accepted?	recognition (compiler)
Does it change what a program means?	the semantic model (compiler)
Does it change what a target emits for the same meaning?	that target's exporter (compiler)
Does it change only pixels, interaction or storage?	the application host

COMMON PITFALL Confusing a schema with data

An entity in this language is a *kind of thing* (Account), not a record. The graph has no record-level information. Any feature that needs to know about individual records, such as counting or sampling, is outside the language and needs a different model.

CHECK YOUR UNDERSTANDING

1. Write the graph $G = (V, F, E)$ for the two-line example in chapter 22.
2. In the figure, what is the in-degree of Contact, and what does a high in-degree tell a modeller?
3. Give an example of a structural fact and of a fact about data, and say which one the DSL can express.

6

Lexical analysis: scanning the text

The language is line-oriented. Blank lines and lines beginning with `#` are ignored. Recognition uses narrow patterns:

```
const entityMatch = line.match(/^entity\s+(\w+)\s*(?:\s*(.))?$ /i);
const relMatch = line.match(/^(\w+)\.(\w+)\s*(=>|~>|->)\s*(\w+)\s*$/i);
```

The first captures an entity and optional field list; the second captures child entity, child field, operator, and parent entity. Because declarations do not span lines, classification and extraction can safely occur in one operation. Bracket metadata such as `AnnualRevenue[Currency, Required]` is then split into reserved markers and display type text.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: regular expressions as the recogniser

The chapter recognises lines with two regular expressions. That choice has a precise justification in formal language theory. A **regular expression** describes a **regular language**, and Kleene's theorem (1956) shows that regular expressions, finite automata and regular grammars are three descriptions of the same class. Each line kind in this DSL is a regular language, so a single regular expression per kind is exactly enough.

The two productions can be drawn as **railroad diagrams**, which show the shape of a line as a path from left to right.

◆ The entity line as a railroad diagram



FIGURE 5 The entity line as a railroad diagram

◆ The relationship line as a railroad diagram

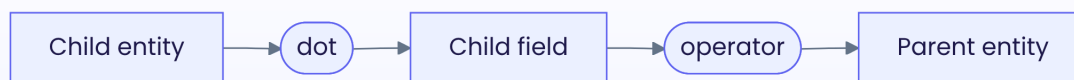


FIGURE 6 The relationship line as a railroad diagram

The field list after the colon is optional; the operator is one of `->`, `=>` and `~>`.

◆ Reading the patterns closely

Every part of a regular expression has a role, and it pays to read each one, because the pattern **is** the definition of the language.

FRAGMENT	MEANING	CONSEQUENCE
<code>^ ... \$</code>	the whole line must match	no stray text before or after
<code>entity</code> with the <code>i</code> flag	the keyword in any case	<code>Entity</code> and <code>ENTITY</code> are accepted
<code>\s+</code>	one or more spaces	a space after the keyword is mandatory
<code>(\w+)</code>	letters, digits and underscore	identifiers are ASCII words only
<code>\s*</code>	optional spaces	<code>Account:Name</code> is accepted
<code>(:\s*(.*))?</code>	an optional colon and the rest of the line	everything after the colon is captured as free text
<code>\.</code>	a literal dot	separates child entity from field
<code>(=> ~> ->)</code>	one of three operators	a fixed, small operator set

Two consequences of these details are easy to miss, and both were confirmed by running the patterns on sample lines.

Identifiers are ASCII words. `\w` matches `[A-Za-z0-9_]`. That covers Salesforce API names, including custom-object suffixes such as `Account__c`, but it also admits names that begin with a digit (`entity1Account` is accepted although the platform would reject such a name), and it excludes spaces and hyphens. The recogniser is a *superset* of the valid names, so the difference must be caught by domain validation (chapter 13).

The anchors define the language, and the two patterns treat trailing text differently. The relationship pattern ends with `\s*$` after the parent name, so a trailing comment makes the line fail. The entity pattern captures the rest of the line with `(.*)`, so the same trailing comment is silently absorbed into the field list:

LINE	RESULT UNDER THE TWO PATTERNS AS WRITTEN
<code>Contact.AccountId -> Account # note</code>	rejected (no match)
<code>entity Account : Name # primary</code>	accepted, with the field text <code>Name # primary</code>

The chapter says only that lines *beginning* with `#` are comments, so neither behaviour is a bug in itself. The lesson is that the asymmetry is a decision nobody made explicitly. It deserves a written policy and a test.

◆ Where a regular expression can go wrong

JavaScript regular-expression engines use **backtracking**: they try one way to match and, on failure, back up and try another. That is convenient and can be catastrophic. If a pattern contains nested or adjacent quantifiers that can split the same text in many ways, the number of attempts grows exponentially with the input length. This failure is called **ReDoS** (regular expression denial of service).

The classic example is `^(\\w+\\s*)+$`. Fed a run of letters ending in a character that cannot match, its running time was measured to grow from about 18 ms at 18 characters to about 600 ms at 26 characters, and the growth is exponential in the length of the input. The two patterns in this chapter are safe for a structural reason: every quantifier is followed by a delimiter that the quantified class cannot match, so there is at most one way to match any line (the regular expression is *unambiguous*). Measured on inputs of a million characters, each pattern rejected the line in under ten milliseconds, which is linear behaviour.

PATTERN FEATURE	RISK	VERDICT HERE
<code>\\w+</code> followed by <code>\\s*</code> or a literal	none, the classes do not overlap	safe
A trailing <code>(.*)</code>	none, it consumes the rest once	safe
A nested quantifier such as <code>(a+)+</code>	exponential backtracking	absent
Alternation of overlapping branches	possible slowdown	absent: the three operators differ

COMMON PITFALL Splitting the field list on commas

The captured field list `Name, AnnualRevenue[Currency, Required]` contains a comma *inside* the brackets. A naive split on every comma produces `AnnualRevenue[Currency` and `Required]` as separate fields. The split must ignore commas inside brackets, which needs only a depth counter that goes up on `[` and down on `]`. The chapter's grammar does not show the bracket production, so a complete grammar (Appendix A) must state it, and a test must cover it.

◆ Two-level recognition

The design is a two-stage recogniser. The first stage (the line patterns) classifies the line and extracts big pieces. The second stage (field splitting and bracket metadata) parses the text captured by `(.*)`. Because declarations never span lines, both happen in one pass. This style is sometimes called *scannerless parsing*: there is no separate token stream, because the tokens are the pieces the regular expressions capture.

CHECK YOUR UNDERSTANDING

1. Predict, then test, what the patterns do with `Contact.AccountId->Account`, `Contact . AccountId -> Account` and `entity Account` (leading spaces).
2. Rewrite the relationship pattern so it accepts a trailing comment. What else must change to keep the two patterns consistent?
3. Why does making the pattern `(\w+\s*)+` dangerous even though it looks harmless?

7

Indentation and tree construction

Indentation has no semantic significance because the grammar has no blocks, nesting, or continuation lines. An AST is likewise omitted: each declaration contributes directly to a flat semantic graph. Building a tree only to flatten it would be unnecessary overhead.

If namespaces, inheritance blocks, aliases, or multiline declarations are added, an AST should be inserted between recognition and semantic analysis. The semantic-model contract can remain stable while the front end becomes more formal.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: when does a language need a tree?

The chapter argues that indentation carries no meaning here and that an AST would be wasted effort. Theory gives a crisp criterion for the general question.

A tree (or any stack-based structure) is needed exactly when the language has **recursive nesting**: constructs that can contain constructs of the same kind, to arbitrary depth. Recognising such a language needs a stack, which is why a *pushdown automaton* is the machine for context-free languages. A parse tree is nothing more than a recording of what that stack did. A language made of independent flat lines has no recursion and needs no stack, so there is nothing for a tree to record.

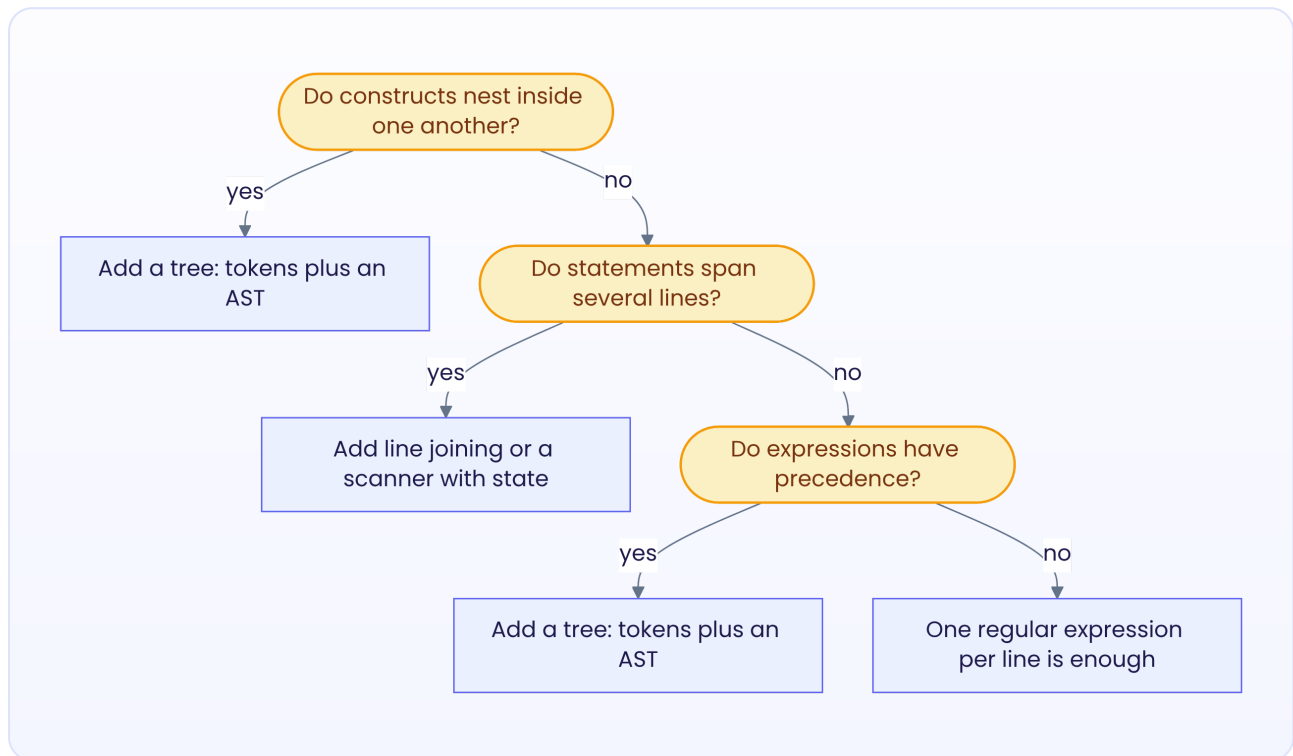


FIGURE 7 Going deeper: when does a language need a tree?

The ER DSL answers *no* three times and lands on the regular-expression rung. If it were to acquire namespaces, inheritance blocks or multiline declarations, the chapter says an AST should be inserted between recognition and semantic analysis, and the flow chart shows why.

◆ Deforestation: not building what you will throw away

The chapter adds a second argument: building a tree only to flatten it again is unnecessary overhead. Functional programming has a name for eliminating such intermediate structures, **deforestation** (Wadler, 1990): if a producer builds a structure that a consumer immediately traverses and discards, fuse the two so the structure never exists. Here the "producer" would be an AST builder and the

"consumer" the code that turns declarations into a flat graph. Because each line contributes independently to the graph, the fused form, recognising a line and updating the model in the same step, is both simpler and faster.

THEORY Concrete syntax, abstract syntax and the model

A compiler with a tree distinguishes three things: the *concrete syntax* (exactly what was typed), the *abstract syntax* (its structure without punctuation) and the *semantic model* (what it means). This DSL collapses the first two: its lines are so simple that their structure is already the model. The moment formatting must be preserved (for example a source formatter that keeps comments), a concrete syntax tree becomes valuable again, because the flat model does not remember comments or spacing.

◆ Layout insensitivity

The chapter says indentation is not significant. In grammar terms the language is **layout-insensitive**: the meaning of a line does not depend on its leading whitespace. For this to be true the classifier must remove leading whitespace before matching, since the patterns begin with `^entity` and `^(\\w+)`. Testing an indented line such as `entity Account` against the patterns as written gives no match, so the guarantee depends on a trim step that the chapter does not describe. It is worth confirming that step, because "indentation is ignored" is a claim that users will rely on.

COMMON PITFALL Half-ignoring layout

A language that ignores indentation must ignore *all* of it: leading spaces, tabs and non-breaking spaces pasted from documents. A line that fails only because a tab was pasted looks identical to a valid one in the editor, which is one of the most frustrating classes of user error.

CHECK YOUR UNDERSTANDING

1. Design a small extension, `namespace Sales { entity Account }` on one line or over several. At which answer in the flow chart does it force a tree?
2. What information does a flat model lose that a concrete syntax tree would keep?
3. Write three test lines that probe the "layout insensitive" claim.

8

Grammar and top-level dispatch

An informal grammar is:

```
program      ::= line*
line         ::= blank | comment | entity | relationship
entity       ::= "entity" identifier (":" field-list)?
relationship ::= identifier "." identifier operator identifier
operator     ::= "->" | "=>" | "~>"
field-list   ::= field ("," field)*
```

Dispatch handles blank/comment lines, then entity syntax, then relationship syntax, and otherwise reports an error. Ordering matters: a broad recognizer placed before a narrow one can introduce ambiguity. A written grammar is valuable even for a small DSL because it defines boundaries and guides negative tests.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: the grammar is regular, and that is a feature

The chapter gives an informal grammar and notes that a written grammar is valuable even for a small DSL. Formal-language theory can now say exactly what *kind* of grammar it is.

All the productions are built from sequences, choices and repetition of terminal symbols and identifiers, with no production that contains itself in the middle of a right-hand side. Such a grammar is **regular**. That means the entire language, "a program is a sequence of lines", is recognised by a single finite automaton, and a single regular expression could in principle describe an entire file.

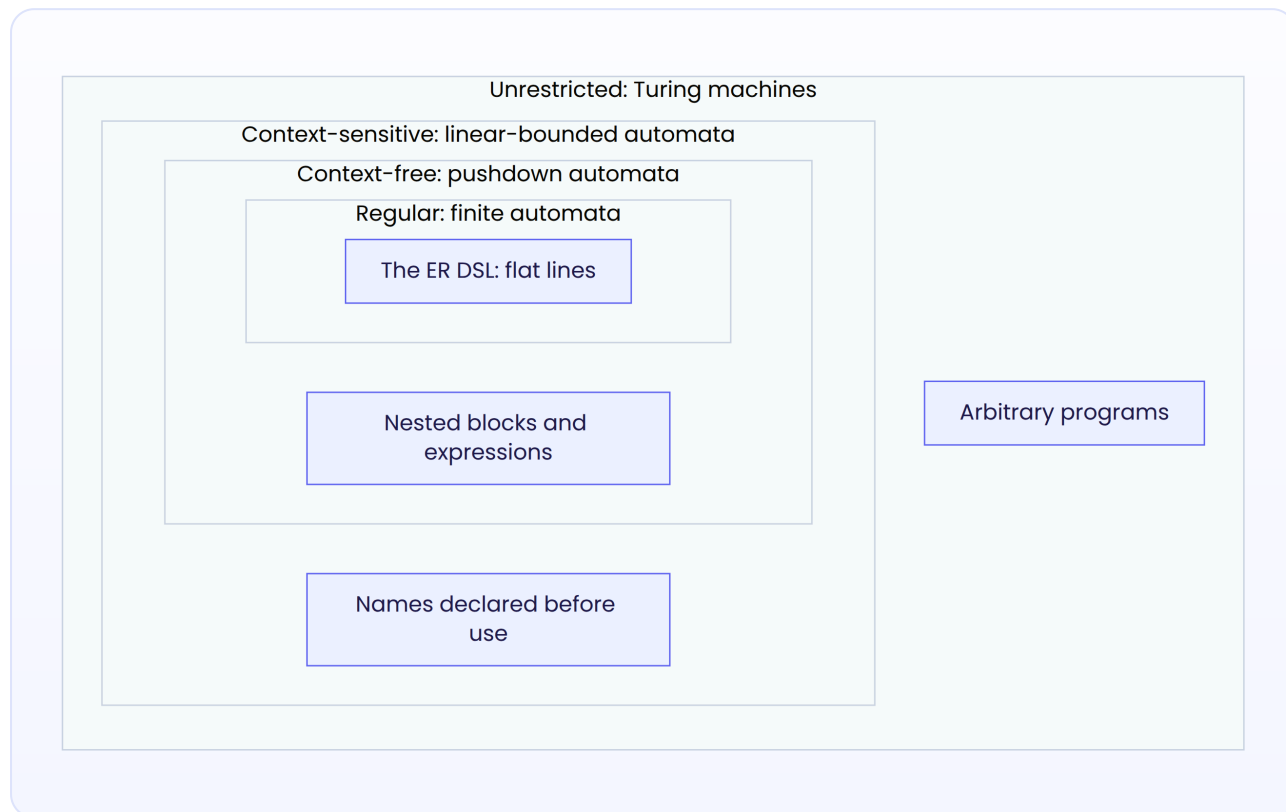


FIGURE 8 Going deeper: the grammar is regular, and that is a feature

This is the smallest of the four classes in the **Chomsky hierarchy** (1956), which is exactly why a regular expression per line is a *sufficient* tool rather than a shortcut. The class of languages with names that must be declared before use is larger, and even that requirement is handled in real compilers by a symbol table rather than by the grammar.

◆ Dispatch is ordered choice

The chapter says dispatch tests blank lines and comments, then entity syntax, then relationship syntax, and otherwise reports an error.

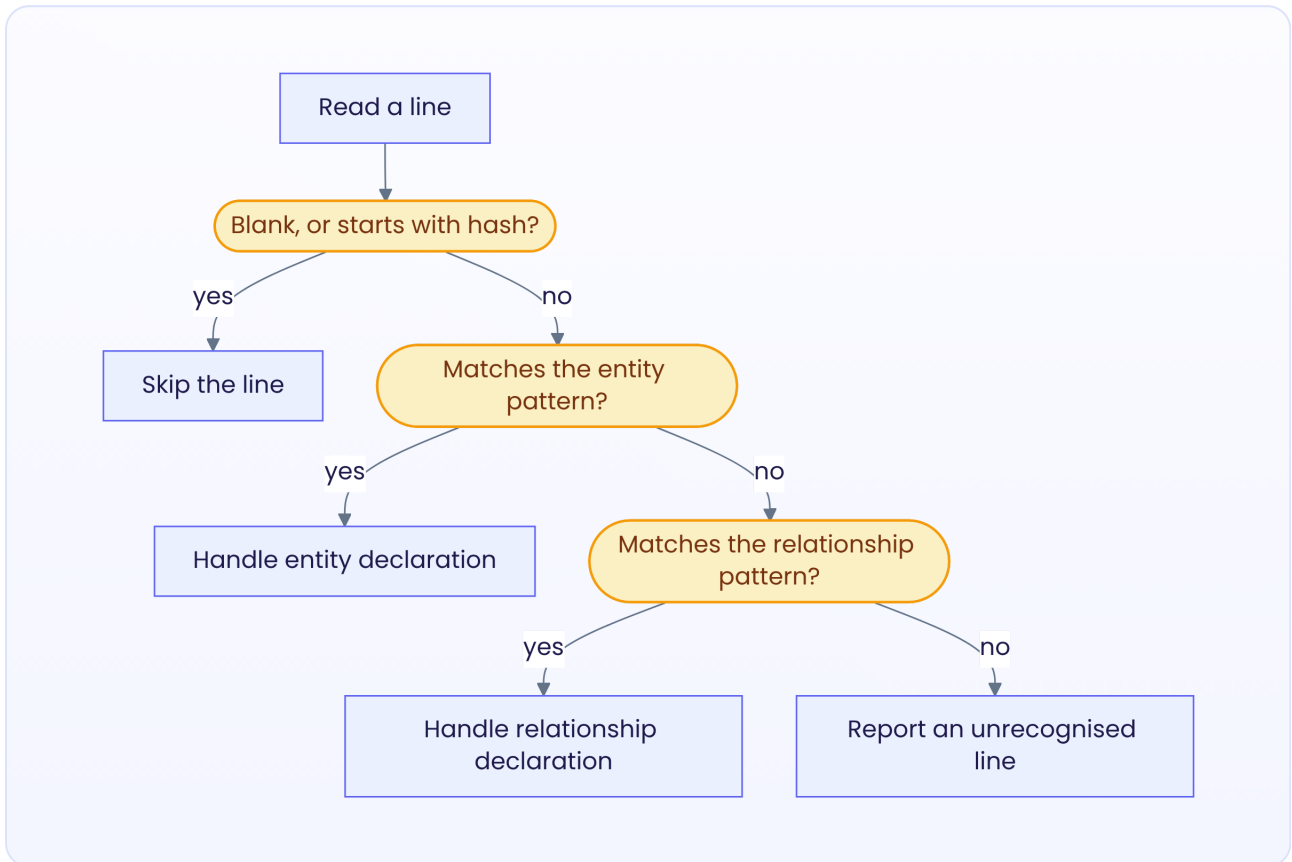


FIGURE 9 Dispatch is ordered choice

"Try this, then that, and take the first that succeeds" is called **ordered choice**, and it is the central idea of *parsing expression grammars* (Ford, 2004). It differs from the unordered alternatives of a context-free grammar in an important way: when two alternatives could both match, ordered choice silently picks the first, while a context-free grammar would call the input **ambiguous**. Here the two alternatives are disjoint, so the order does not affect correctness. That is worth *proving* rather than assuming.

FORMAL VIEW**The entity and relationship languages are disjoint**

Let L_e be the set of lines matched by the entity pattern and L_r those matched by the relationship pattern. A line in L_e starts with the word `entity` followed by at least one space. A line in L_r starts with an identifier immediately followed by a dot. No line can do both, because the character after the first word is either a space or a dot:

$$L_e \cap L_r = \emptyset$$

Consequently the dispatcher is deterministic and the order of the two tests is immaterial. The keyword collision cases confirm this: `entity entity : Name` is an entity declaration (the entity is named "entity"), while `entity.Foo -> Bar` is a relationship whose child is named "entity".

The moment a third line kind is added, this argument must be repeated. If the new pattern is broad, such as "any line with an equals sign", and is placed before a narrow one, it will steal lines from the narrow one. That is the hazard the chapter warns about in one sentence.

◆ Grammar-derived negative tests

A written grammar tells you not only what is valid but where the *edges* of validity lie. Every constraint in a production suggests a test that breaks exactly that constraint.

CONSTRAINT IN THE GRAMMAR	VALID EXAMPLE	NEAR-MISS THAT MUST BE REJECTED
a dot between entity and field	<code>Contact.AccountId -> Account</code>	<code>Contact AccountId -> Account</code>
exactly one operator of three	<code>Contact.AccountId => Account</code>	<code>Contact.AccountId --> Account</code>
a parent entity after the operator	<code>Contact.AccountId -> Account</code>	<code>Contact.AccountId -></code>
a space after the keyword	<code>entity Account</code>	<code>entityAccount</code>
the parent is a bare identifier	<code>Contact.AccountId -> Account</code>	<code>Contact.AccountId -> Account.Name</code>
a single dot	<code>Contact.AccountId -> Account</code>	<code>Contact.Account.Id -> Account</code>

All of the rejected lines in the table were confirmed to produce no match under the two patterns as written.

◆ Recogniser versus domain

There is one more use of a written grammar. By comparing the *set of strings the grammar accepts* with the *set of strings the domain considers valid*, you find the gap. The identifier rule accepts `1Account` and `___` although neither is a valid Salesforce API name. The grammar is a **superset** of the domain, and the difference is exactly the work of semantic validation.

CHECK YOUR UNDERSTANDING

1. Explain why adding a line kind for comments at the end of a line, such as `Name # note`, could break the disjointness proof.
2. Convert the ordering in the dispatch flow chart into a table showing which lines each stage claims.
3. Write the smallest new production that would make the language non-regular, and say what machinery it would then require.

9

Recursive descent parsing for conditions

Recursive descent is not applicable because the DSL has no boolean expressions, predicates, precedence, or nested condition nodes. The lesson is methodological: parsing strategy should follow grammar structure, not compiler fashion.

If expressions are added later, parse them into an AST with explicit precedence and analyze them separately. Do not encode nesting in increasingly complex regular expressions.

◆ Going deeper: choosing a parsing strategy from the grammar

The chapter is a *negative* chapter: recursive descent does not apply, because the language has no expressions. Its lesson, that parsing strategy should follow grammar structure and not compiler fashion, can be turned into a decision procedure.

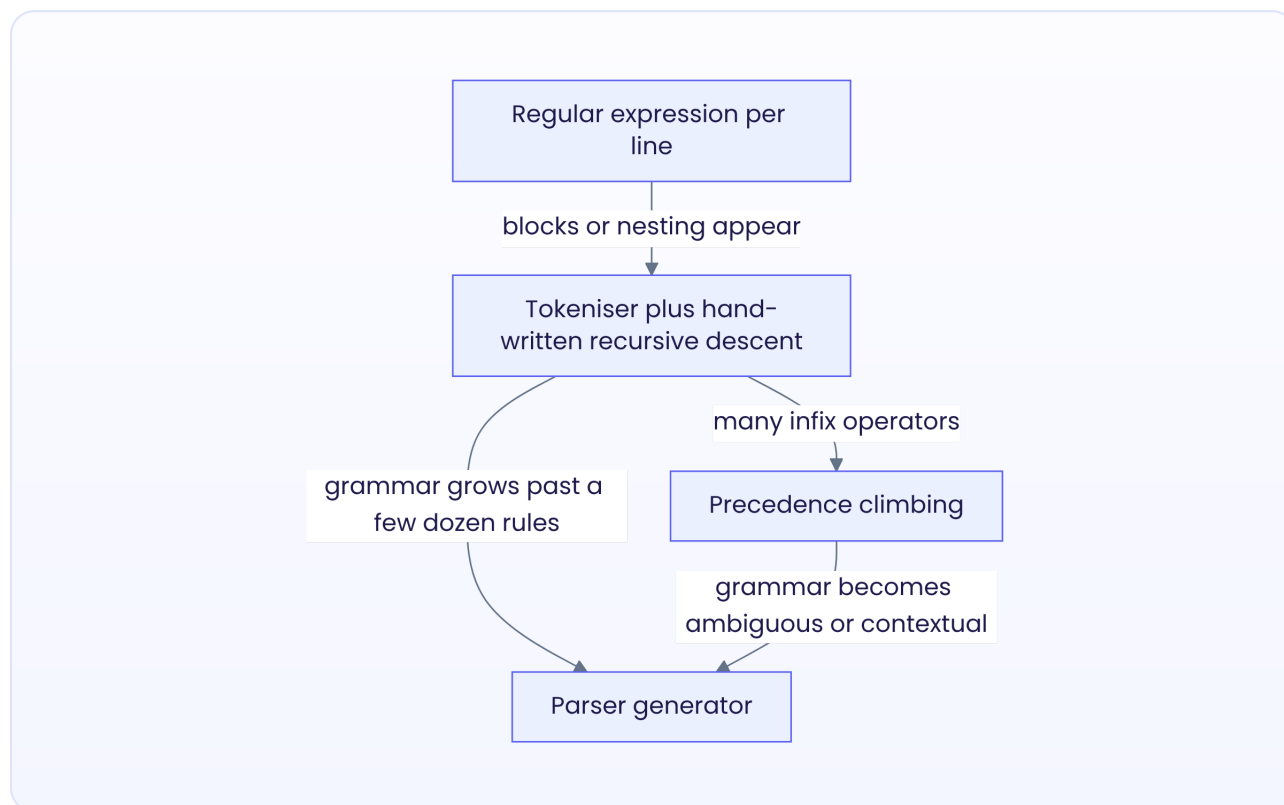


FIGURE 10 Going deeper: choosing a parsing strategy from the grammar

RUNG	WHEN IT IS THE RIGHT TOOL	WHAT IT COSTS
Regular expression per line	lines with a fixed shape and no nesting	almost nothing; this DSL
Tokeniser and recursive descent	nested structure, blocks, parenthesised expressions	a scanner, a parser, an AST
Precedence climbing	several operators with different precedence	a table of binding powers
Parser generator	large or evolving grammars	a build step, harder error messages

◆ Why regular expressions cannot count

The chapter warns against encoding nesting in "increasingly complex regular expressions". The reason is a theorem, not a preference. A finite automaton has finitely many states, so on a long enough input it must revisit a state, after which it cannot know how many symbols it has already seen. This idea, the *pumping lemma*, shows that no regular expression can recognise all correctly balanced parentheses. Recognising nesting needs unbounded memory: a stack.

Some regular-expression dialects add recursion or balancing groups, which lets a pattern match nested structure. Such patterns are no longer regular expressions in the formal sense, they are hard to read, and their failure modes are hard to test. The following **smells** indicate that a regular expression is being stretched past its purpose:

SMELL	WHY IT SIGNALS TROUBLE
a lookahead used to count or balance	counting needs a stack
a loop that repeatedly strips the innermost parentheses	manually simulating recursion
a pattern longer than a screen	no one can review it
several nested optional groups	ambiguity and slowdown risk
a comment explaining what the pattern "really" does	the pattern is carrying meaning that code should

◆ What a future filter language would look like

Suppose the DSL gained diagram filters as source syntax, for example `show Account, Contact where Type = Customer and not Archived`. The moment `and`, `not` and parentheses appear, the language has nested structure and precedence, and the first two rungs of the ladder are crossed. Its grammar, using the stratification technique that gives `not` the strongest binding and `or` the weakest, would look like this:

```
filter = orExpr
orExpr = andExpr { "or" andExpr }
andExpr = notExpr { "and" notExpr }
notExpr = "not" notExpr | atom
atom = field comparison value | "(" orExpr ")"
```

The companion guide on the SML compiler builds exactly this machinery (a scanner, a recursive-descent parser and an evaluator) for user conditions, and it can be read as a worked template if the ER DSL ever needs it. The important design rule is the one the chapter gives: parse expressions into an AST with explicit precedence and analyse them separately, and do not keep extending the line regular expressions.

COMMON PITFALL The regular expression that grows

Each small addition to a pattern, an optional group here and an alternative there, is easy to justify. After a dozen of them the pattern accepts strings nobody intended and rejects strings nobody tested. Set a rule in advance, for example "if a pattern needs a fourth capture group, or a lookahead, we write a tokeniser".

CHECK YOUR UNDERSTANDING

1. Choose the rung of the ladder for each: a config file with `key = value` lines, a language with `if / else` blocks, a formula language with `+ - * /` and brackets.
2. Explain the pumping-lemma idea in two sentences to a colleague.
3. Which smell in the table would you find first in a pattern that matches `(a(b)c)` style nesting, and what would you do?

10

Name resolution and symbol tables

Entities are indexed case-insensitively while preserving first-seen display casing:

```
JAVASCRIPT

const ensureEntity = (name) => {
  const key = name.toLowerCase();
  if (!entities.has(key)) entities.set(key, { name, fields: [] });
  return entities.get(key);
};
```

This establishes identity semantics: `Account`, `account`, and `ACCOUNT` denote one entity. Relationship targets can be referenced before declaration; implicit creation is a deliberate policy that supports forward references. A strict mode could instead diagnose undeclared targets.

A symbol table defines identity, lookup, normalization, and scope. This DSL has one global namespace; larger languages may require nested symbol tables.

◆ Going deeper: identity, interning and forward references

The chapter's `ensureEntity` function is a small piece of theory in disguise. Its key line, `name.toLowerCase()`, defines what it means for two names to be **the same entity**.

FORMAL VIEW Identity as an equivalence relation

Define two names to be equivalent when they are equal after lower-casing:

$$a \sim b \Leftrightarrow \text{lower}(a) = \text{lower}(b)$$

This relation is *reflexive*, *symmetric* and *transitive*, so it partitions all names into **equivalence classes**: `Account`, `account` and `ACCOUNT` fall in one class. The map's keys are the classes, and the stored `name` is a chosen **representative** of the class. The function `lower` is called a *normal form* or *canonical form*, and it is idempotent (`lower(lower(a)) = lower(a)`), which is what makes repeated normalisation safe.

The function then does what compiler writers call **interning** (or *hash consing*): the first request for a class creates the object, and every later request returns *the same object*. After interning, identity can be tested by reference and any later stage can rely on one object per entity. It is the reason the chapter can say declarations and relationships "refer to one stable object".

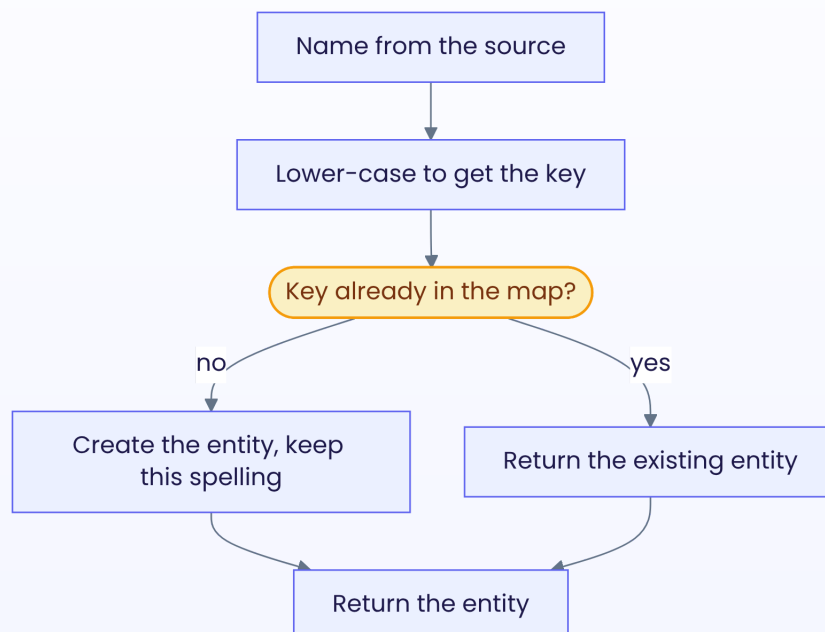


FIGURE 11 Going deeper: identity, interning and forward references

◆ The ASCII restriction removes a notorious problem

Case-insensitive comparison is easy for ASCII text and famously subtle for the rest of Unicode. The Turkish dotted and dotless *i* means that a locale-sensitive lower-casing of `I` gives a different letter, and some letters have no single-character lower-case form. The recogniser's `\w` matches only ASCII letters, digits and underscore, so the names that reach `toLowerCase()` are already in the safe range. This is a quiet design benefit of the narrow pattern. It also means that if identifiers are ever widened to allow international names, the identity rule must be revisited (normalise the text first, then use full case folding).

◆ Forward references and implicit declaration

Relationship targets can be referenced before they are declared, and an undeclared target is created implicitly. This is called **declaration by use**, and it has a long history, including the implicit typing of variables in early Fortran. Its consequences are well known.

POLICY	ADVANTAGE	DISADVANTAGE	ANALOGY
Implicit creation (this DSL)	order-independent; convenient for imported metadata	a typo silently creates a new entity	early Fortran, permissive scripting
Warning on undeclared use	keeps order-independence, flags typos	needs a diagnostics channel	linters
Error on undeclared use (strict mode)	catches all typos	forces declarations before or after use	<code>IMPLICIT NONE</code> , JavaScript strict mode

The chapter suggests a strict mode as a future option, and the right way to offer it is as a *setting*, because both behaviours are legitimate. Logically, implicit creation follows an **open-world** attitude (the model may contain things not yet described), and strict mode a **closed-world** attitude (whatever is not declared does not exist). Chapter 13 develops this contrast.

◆ Which spelling is the representative?

The chapter says the first-seen spelling is preserved. That is a *choice function* over the equivalence class, and it has a consequence worth stating: **the displayed name can depend on the order of the lines**. If a relationship refers to `account` before the declaration `entity Account`, the entity will be displayed as `account`. Only the display changes, not the identity, but it can surprise users.

POLICY FOR THE DISPLAY SPELLING	DEPENDS ON LINE ORDER?	BEHAVIOUR
First seen (this DSL)	yes	the earliest spelling wins
Declaration wins over references	no	the spelling in <code>entity X</code> is used when present
Canonical from imported metadata	no	the platform's own casing is used
Lexicographically smallest	no	deterministic but often ugly

Case-preserving, case-insensitive naming is what most file systems do, so users tend to find it natural. The design task is only to make the tie-breaker deterministic and documented.

◆ Two namespaces

Entities live in one global namespace. Fields live in a **second, per-entity** namespace: a field name is unique only within its entity, and the full name is the pair (*entity*, *field*). The chapter notes that fields are found by scanning arrays, which is fine for small entities. Two points follow. Field identity should use the *same* normalisation as entity identity, otherwise `AccountId` and `accountid` become two fields. And the array scan costs $O(k)$ per lookup for an entity with k fields, which matters only when entities have hundreds of fields.

WORKED EXAMPLE A typo and what would catch it

With implicit creation, `Contact.AccountId -> Acount` creates a second entity named `Account` next to `Account`. A strict mode would report it. A friendlier alternative is a suggestion based on **edit distance**, the minimum number of insertions, deletions and substitutions needed to turn one string into another (Levenshtein, 1965). The distance between `Account` and `Account` is 1, so the message can say: *Undeclared entity "Account". Did you mean "Account"?*

COMMON PITFALL Comparing display names

If any code compares `entity.name` values instead of normalised keys, `Account` and `account` are treated as different again. Provide one function that computes the key and use it everywhere, including the editor's completion and the diagnostics.

CHECK YOUR UNDERSTANDING

1. Show that the relation $\sim$ is transitive using the definition.
2. Design a small rule so the displayed spelling is independent of the order of lines.
3. What should happen when `Account` is later declared with `entity Account`? Is the phantom entity an error or a feature?

11

Stateful interpretation and safe mutation

`parseEr()` incrementally updates an entity map, field arrays, relationship records, and a deduplication set. `ensureField()` creates a field once and merges later metadata, so entity declarations and relationship declarations refer to one stable object.

Exact duplicate relationships are identified by child entity, child field, parent entity, and kind. Controlled mutation is acceptable internally, but the returned model should have stable defaults such as `isRelationship`, `relatesTo`, `isRollupSummary`, `isRequired`, and `dataType`. Stable shape reduces downstream defensive code.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: parsing as a fold over independent facts

The chapter describes `parseEr()` as incrementally updating an entity map, field arrays, relationship records and a deduplication set. In the language of functional programming this is a **fold**: a state is threaded through the lines, and each line updates it.

FORMAL VIEW The parser as a fold

Let the state be a triple $s = (E, R, D)$ holding the entity map, the relationship list and the set of keys already seen. Then:

$$\begin{aligned} \text{parseEr}(\text{text}) &= \text{finalise}(\text{foldl step } s_0(\text{lines}(\text{text}))) \\ s_0 &= (\emptyset, [], \emptyset) \end{aligned}$$

Each call of *step* handles one line: it may add entities and fields, add a relationship unless its key is already in *D*, or do nothing for blanks and comments. The function *finalise* fills in the stable defaults the chapter lists.

◆ Algebraic laws worth having

Because each line contributes independently, the fold has properties that can be stated, and each is also a test.

LAW	STATEMENT	WHY IT MATTERS
Idempotence	repeating a line changes nothing	duplicates are harmless, exact duplicate edges are not doubled
Order independence of meaning	permuting lines gives the same entities, fields and edges	the language is declarative
Order dependence of presentation	permuting lines may change display spelling and drawing order	a known, documented effect
Comment neutrality	adding blank or comment lines changes nothing	layout does not matter
Chunking	parsing the two halves of a file separately and combining the models equals parsing the whole	enables incremental work

The set semantics of the deduplication step implements the first law. The chapter says duplicates are identified by child entity, child field, parent entity and kind. Two details deserve a decision. **Keys must be normalised:** `Contact.AccountId -> Account` and `contact.accountid -> account` are the same edge, so the key must use lower-cased names. And **the kind is part of the key**, so the same field declared with `->` and with `=>` produces two edges. A relationship field is normally one kind or the other, so this is a semantic conflict that exact-duplicate detection will not catch. It is a natural warning to add.

◆ Merging field metadata

`ensureField()` creates a field once and merges later metadata. Whether merging is safe depends on the algebra of the merge.

ATTRIBUTE	MERGE RULE	COMMUTATIVE?	IDEMPOTENT?	COMMENT
Markers such as required	set union	yes	yes	safe to merge in any order
Relationship flag	logical or	yes	yes	once a relationship field, always one
Display type text	first wins, last wins, or non-empty wins	depends	yes	order-dependent unless conflicts are diagnosed

A merge that is commutative, associative and idempotent forms a **semilattice**, and for such merges the order of declarations cannot matter. Markers and flags satisfy this. The display type does not unless a policy is chosen, for example: keep the first non-empty type and report a warning if a later one disagrees. The point is not that the current behaviour is wrong, but that the *policy should be stated*.

◆ Controlled mutation behind an immutable interface

The chapter accepts mutation inside `parseEr()` but wants the returned model to have a stable shape. This is a well-known compromise: mutate freely **inside** a function whose result is a fresh value, and treat the result as immutable from the outside. Because the maps and arrays never escape while they are being built, no caller can observe a half-built state. A development-time `Object.freeze` on the returned model can turn accidental mutation by a renderer into an immediate error.

A stable shape helps twice. It removes defensive code (`field.dataType ?? ''`) from every consumer, and JavaScript engines optimise objects that always have the same properties in the same order, so uniform shapes are also faster in practice.

COMMON PITFALL Half-built defaults

If `isRequired` is only set when a marker is present, some fields carry the property and some do not. Consumers then need the same defensive check everywhere, and a missed one is a `undefined` in an export. Fill every default in one place, in `finalise`, so that a field of the returned model always has every documented property.

CHECK YOUR UNDERSTANDING

1. State the chunking law formally and say what "combine" must mean for it to hold.
2. Which line order would produce the display spelling `contact` for an entity written elsewhere as `Contact` ? Do you consider that acceptable?
3. Propose a test that detects a field appearing twice under two spellings.

12**Intermediate representation and step generation**

The semantic model is the primary IR:

```
{ entities: [{ name, fields: [...] }],  
  relationships: [{ childEntity, childField, parentEntity, kind }] }
```

It stores meaning rather than pixels. `buildErGeometry()` creates a second IR containing boxes, rows, paths, lane assignments, and SVG dimensions. Mermaid bypasses this IR because it has its own layout engine; canvas and draw.io require it.

Separate IRs localize failures: semantic errors can be tested without a browser, routing errors without reparsing, and escaping errors within one backend.

◆ Going deeper: two IRs and what each one is for

The chapter identifies the semantic model as the primary IR and the geometry as a second. The two have different jobs, and the difference is what theory calls **levels of representation**: a high-level IR that keeps the meaning of the source and hides everything about the output, and a lower-level IR that has committed to a particular way of drawing.

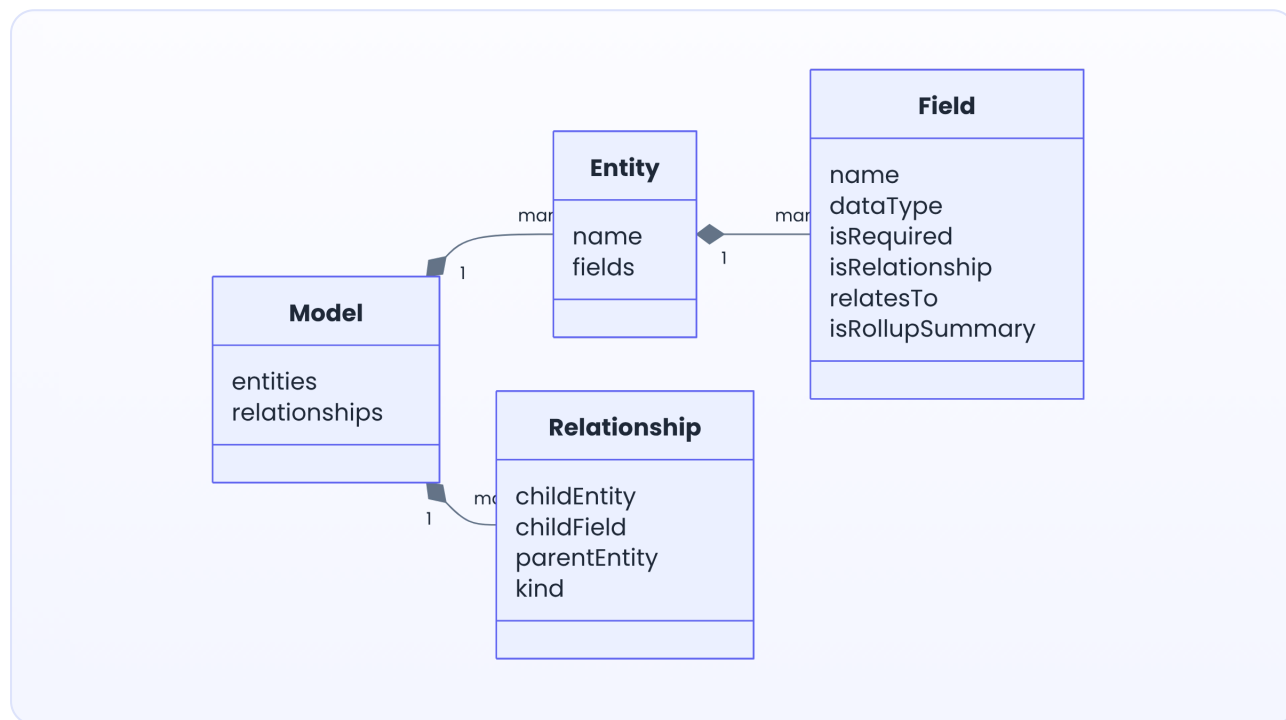


FIGURE 12 Going deeper: two IRs and what each one is for

The class diagram shows the semantic model as the chapter describes it, with the stable per-field defaults. It contains **no** coordinates, sizes or colours. The geometry IR is a different kind of object.

◆ The geometry IR as a class diagram

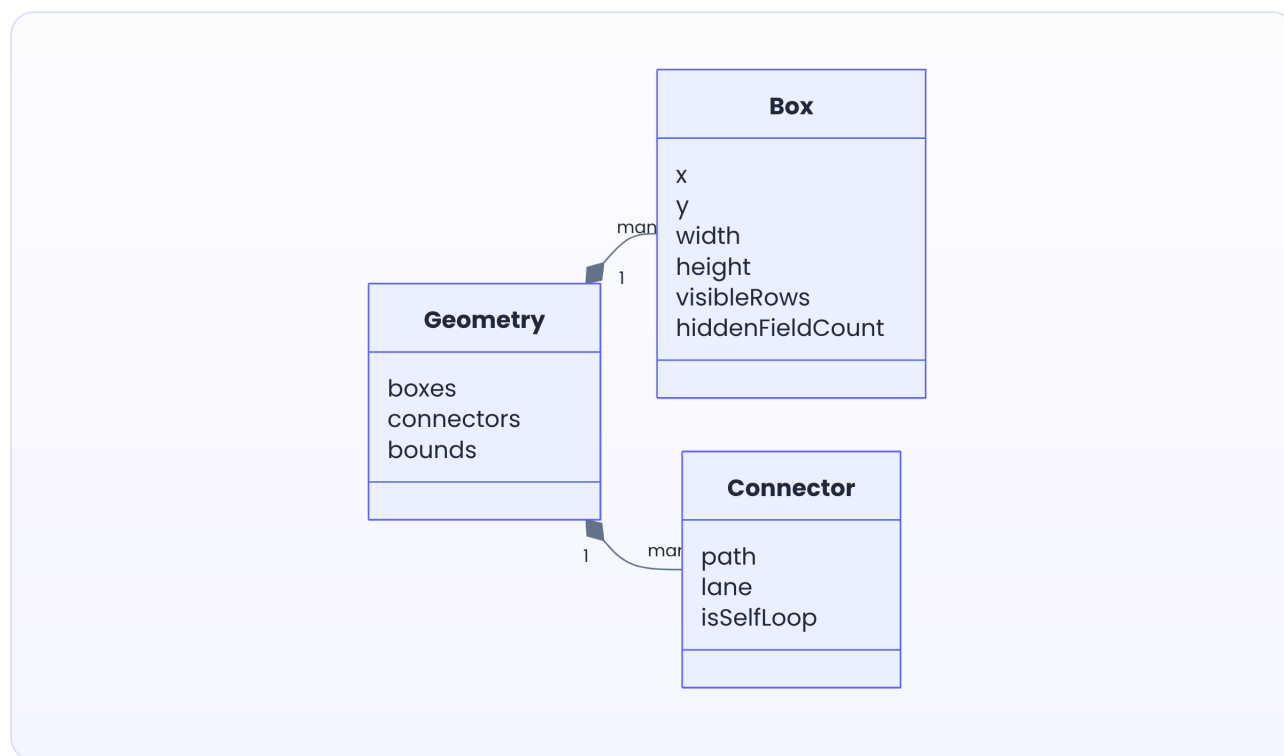


FIGURE 13 The geometry IR as a class diagram

PROPERTY	SEMANTIC MODEL	GEOMETRY IR
Level	high: independent of any output	mid: committed to a layout
Answers	what exists and how it relates	where things go and how they are drawn
Consumers	every backend	canvas and draw.io
Changes when	the source text changes	the source, the overrides or the design rules change
Testable without	a browser and a layout engine	a browser and a parser

Because each box and connector should record which entity or relationship it came from, geometry supports **provenance**: a click on the canvas can be traced back to the model. Stable identifiers make that link reliable.

◆ Layout as a graph-drawing problem

Turning a graph into a picture is a research field of its own. The standard approach for directed graphs is the **layered layout** of Sugiyama and colleagues (1981), which proceeds in phases: remove cycles by reversing some edges, assign each vertex to a layer, order the vertices within layers to reduce edge crossings, and finally assign coordinates. Mermaid performs a layout of this kind itself, which is why the chapter says it bypasses the geometry IR. The Studio's own geometry builder takes a simpler route, placing boxes and honouring saved positions, and then solves several smaller problems that appear in the list of geometry contents.

Parallel edges and lanes. If k edges connect the same pair of boxes, drawing them on top of each other hides all but one. The standard fix assigns each edge its own *lane*, offsetting them symmetrically around the centre line.

FORMAL VIEW Lane offsets for parallel edges

For k edges, indexed $i = 0 \dots k-1$, and a lane spacing δ , the offset of edge i from the centre line is

$$\text{offset}(i) = (i - (k - 1) / 2) \times \delta$$

For $k = 1$ the offset is 0 (a single edge stays centred). For $k = 3$ the offsets are $-\delta$, 0 and $+\delta$. The construction is symmetric, so the picture does not lean to one side.

Self-loops. An edge from an entity to itself cannot be a straight line between two different boxes. It needs a small loop that leaves one side of the box and returns to it. The requirement stated in the chapter's test list, "non-degenerate self-loops", means the loop must have a positive size, otherwise it collapses to a point and disappears.

Canvas bounds and clamping. Clamping a coordinate to a range is the function $\text{clamp}(x, lo, hi) = \min(\max(x, lo), hi)$. Applied to box positions it prevents saved coordinates from placing a box outside the visible canvas.

Hidden fields. If an entity has n fields and the box can show at most m rows, the box shows m rows and a counter of $n - m$ hidden fields. This is a small example of **level of detail**: show the essential part and summarise the rest.

◆ Stable identifiers

Both the draw.io backend and any saved layout need identifiers. There are two common schemes.

SCHEME	EXAMPLE	STABLE WHEN LINES ARE REORDERED?	STABLE WHEN A NAME CHANGES CASE?
By position	entity-0 , entity-1	no	yes
By normalised name	entity-account	yes	yes
By content hash	a hash of the normalised name	yes	yes

Identifiers derived from **normalised identity** are the safest: they survive reordering and cosmetic edits, which keeps saved positions and diffs meaningful.

COMMON PITFALL Coordinates in the model

When the two IRs are blurred, the model starts to carry positions "for convenience". Then the source is no longer the only thing that determines the model, and tests of semantics need geometry set-up. Keep the levels apart.

CHECK YOUR UNDERSTANDING

1. For 4 parallel edges and $\delta = 12$, compute the lane offsets.
2. Which IR would you inspect first for a bug in which a connector crosses the wrong box, and why?
3. Suggest two properties that any valid geometry should satisfy, beyond positive dimensions.

13

Semantic validation and domain-aware checks

Validation has lexical, syntactic, structural, domain, and target-aware levels. The parser validates line shape, normalizes case, creates missing relationship endpoints according to policy, preserves field metadata, rejects unknown top-level syntax, and deduplicates exact edges.

Unknown bracket content is retained as display metadata rather than checked against a closed type vocabulary. This distinction between preservation and type checking is important for imported Salesforce metadata. Every permissive behavior should be documented and tested; strict and permissive modes may be useful later.

◆ Going deeper: levels of validity, open and closed worlds

The chapter lists five levels of validation. Each answers a different question and belongs to a different stage.

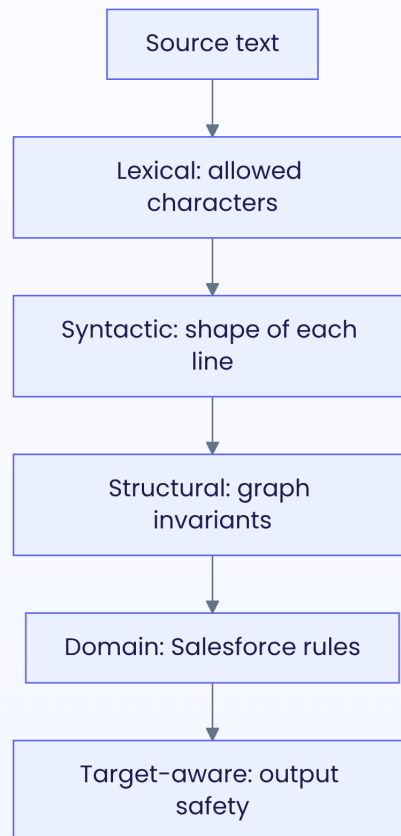


FIGURE 14 Going deeper: levels of validity, open and closed worlds

LEVEL	QUESTION	EXAMPLE CHECK	TODAY
Lexical	Are the characters allowed?	identifiers are word characters	in the patterns
Syntactic	Does the line have a valid shape?	dot, operator and parent are all present	in the patterns
Structural	Do the graph invariants hold?	one entity per identity, no duplicate edges	in the parser
Domain	Is it valid for Salesforce?	names start with a letter, types are known	mostly absent (preserved)
Target-aware	Is the output valid in the target?	identifier-safe types for Mermaid	in the exporters

◆ Structural invariants

The structural level is where the model is checked against its own definition. Stating the invariants precisely turns them into tests (chapter 21).

FORMAL VIEW

Invariants of a well-formed model

- (I1) every relationship names a child and a parent that are entities in V
- (I2) no two entities in V have equivalent names
- (I3) no two relationships have the same (child, field, parent, kind), up to equivalence
- (I4) for every relationship, the child field exists and is marked as a relationship field
- (I5) every field carries all documented default properties

Invariant I1 is **referential integrity**: an edge never points at nothing. The chapter's policy of creating missing endpoints implicitly is what makes I1 true by construction. Invariant I4 records a **redundancy**: the model stores the relationship both as an edge and as a marker on the field. Redundant data can drift, so I4 should be checked whenever either representation changes.

◆ Open versus closed worlds

The chapter says unknown bracket content is retained as display metadata instead of being checked against a fixed vocabulary of types. In logic this is the distinction between the **open-world assumption** and the **closed-world assumption**. Under a closed world, anything not in the vocabulary is false (an error). Under an open world, anything not known is merely *unknown*, so it is kept and passed along.

	CLOSED WORLD (STRICT)	OPEN WORLD (PERMISSIVE, CURRENT)
Unknown type <code>Geolocation</code>	error	preserved as display text
Good for	catching typos in hand-written source	importing metadata the DSL has not seen yet
Risk	rejects valid new platform features	accepts typos silently

This is the same trade-off seen in network protocols as the **robustness principle** ("be conservative in what you send, liberal in what you accept") and in data formats as the **tolerant reader** pattern, where a reader ignores or preserves fields it does not understand so that newer data still works with older tools. The counterweight is that leniency spreads. Once users depend on a permissive behaviour, it becomes part of the language whether or not it was intended, which is why the chapter insists that every permissive behaviour be documented and tested.

◆ Preservation is not checking

The distinction the chapter draws between *preserving* and *type-checking* has a precise value for imports. Metadata arriving from a platform describe call may contain types the DSL knows nothing about. Preserving them keeps the round trip loss-free: `import → DSL → parse → export` returns the same text a tool would have produced. Rejecting them would make the compiler fail on a newer platform release. A registry of known types (chapter 24) can then add *warnings* without becoming a gate.

◆ Candidate validations to add

VALIDATION	LEVEL	SUGGESTED SEVERITY
Name does not start with a digit	domain	warning (accepted by the recogniser)
Same field declared with two different kinds	structural	warning
Conflicting display types for one field	structural	warning
Relationship to an undeclared entity	structural	information, or error in strict mode
Unknown marker or type text	domain	hint, since it is preserved
Entity with no fields and no relationships	structural	hint

COMMON PITFALL Validating in the wrong place

Domain rules put into the recogniser make the grammar unreadable, and structural rules put into an exporter get duplicated in each backend. Keep each check at the earliest stage that has the information it needs, and no earlier.

CHECK YOUR UNDERSTANDING

1. Which invariant would be broken if the parser created a relationship without creating its parent entity?
2. Decide, for each row of the candidate validations, whether you would make it strict-only or always on.
3. Give an example where being liberal now creates a compatibility problem later.

14

User conditions vs. record criteria: two expression languages

This compiler does not parse filters or predicates. A condition language has truth values, operators, precedence, null semantics, and coercion; the ER DSL has declaration semantics. These should not be conflated.

Diagram filtering should normally remain a view concern. If filters become source syntax, they require an explicit grammar and semantic model rather than embedded JavaScript expressions.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: what a condition language would demand

The chapter explains that the compiler does not parse filters or predicates, and that diagram filtering should normally remain a view concern. Here is why the theory agrees.

A condition language brings a set of obligations that a declaration language does not have: *truth values*, *operators*, *precedence*, *null semantics* and *type coercion*. Each has to be specified, implemented and tested. None of them is needed to describe a schema.

◆ Filtering is a query over the model

A **view** in database theory is a query defined over base data. Filtering a diagram is exactly that. Given a predicate p over entities, the filtered diagram is the **induced subgraph**: keep the entities that satisfy p , and keep the relationships whose two ends both survive.

FORMAL VIEW

A filtered view as an induced subgraph

$$V' = \{ v \in V : p(v) \} \quad E' = \{ e \in E : \text{source}(e) \in V' \wedge \text{target}(e) \in V' \}$$

In relational algebra, keeping the entities that satisfy p is a **selection** (σ), and hiding some fields of the surviving entities is a **projection** (π). The result depends on the model and the predicate and requires no change to the model itself.

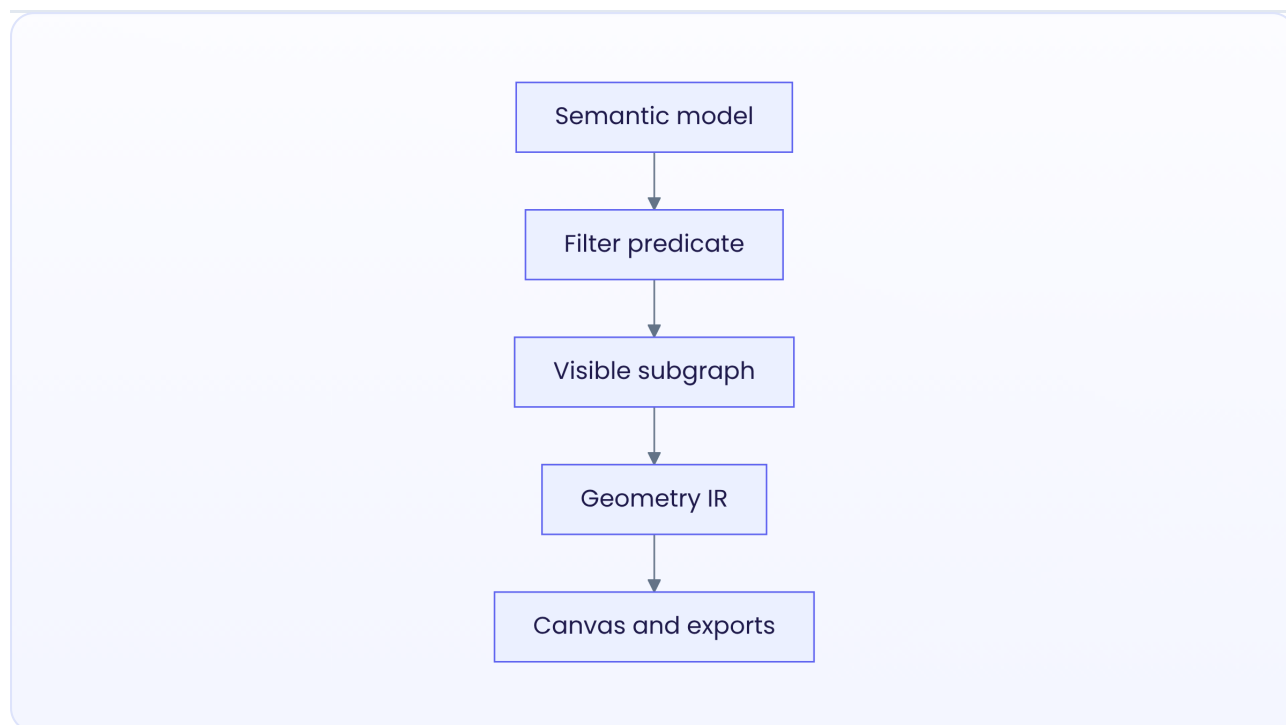


FIGURE 15 Filtering is a query over the model

The figure shows the correct position of a filter: **between** the model and the geometry, not inside the source text. Placing it there keeps the model authoritative, and lets the same source produce many views.

KIND OF FILTER	DESCRIPTION	GRAPH OPERATION
By name	entities whose name matches	selection
By marker	entities with required or rollup fields	selection over fields
Neighbourhood	an entity and everything within k relationships	breadth-first search to depth k
Hide fields	show only relationship fields	projection

◆ If filters ever become source syntax

The chapter says that if filters do become part of the language they need an explicit grammar and semantic model, and not embedded JavaScript expressions. The reasons are strong.

EMBEDDED JAVASCRIPT EXPRESSION	EXPLICIT FILTER LANGUAGE
can do anything the host can do, including side effects	can only express selections
may not terminate	always terminates
unsafe to run on imported or shared source	safe to run on untrusted source
hard to analyse or to translate to other targets	analysable, translatable

This is the **rule of least power** again: use the weakest language that expresses what is needed, because weaker languages can be analysed, secured and ported.

◆ Null and truth values

If conditions do appear, the question of missing values arrives with them. What is the truth value of a comparison with a missing field? Databases answer with a third value, **unknown**, giving *three-valued logic*.

P	Q	P AND Q	P OR Q	NOT P
true	unknown	unknown	true	false
false	unknown	false	unknown	true
unknown	unknown	unknown	unknown	unknown

A filter keeps an entity only when the condition is definitely true. The choice must be made once and applied everywhere, and it is a good reason to keep conditions out of the language until they are needed.

COMMON PITFALL Filtering that changes the model

A filter implemented by deleting entities from the model, for example to speed up drawing, silently changes what every other target emits. Filter a *copy* or a *derived view*, never the authoritative model.

CHECK YOUR UNDERSTANDING

1. Compute the induced subgraph for the schema in chapter 5 if the predicate keeps only Account and Contact.
2. How would a "two relationships away from Opportunity" view be computed?
3. Which of the drawbacks of embedded JavaScript expressions is most serious for a tool that imports shared source?

15

Applying steps and graph expansion

The compiler does not execute a program step by step. A relationship declaration creates an edge; it does not trigger traversal or computation. Layout may traverse edges to route connectors, but that is a rendering algorithm, not source execution.

Future dependency or impact analysis should be implemented as explicit analyses over the semantic model, with derived outputs rather than hidden semantic mutations.

◆ Going deeper: no steps, but many graph algorithms

The chapter states that the compiler does not execute a program step by step: a relationship declaration creates an edge and triggers nothing. This is the defining feature of a declarative language, and it changes what "analysis" means. Instead of tracing an execution, you **query a structure**.

A helpful way to think about it is the logic-programming view. Each declaration is a *fact*. In the notation of Datalog, a small query language for exactly this situation, the facts are rows of a relation, and an analysis is a rule over them:

```
rel(Contact, AccountId, Account, lookup).           % a fact from a declaration
dependsOn(X, Y) :- rel(X, _, Y, _).                 % X refers directly to Y
dependsOn(X, Z) :- dependsOn(X, Y), dependsOn(Y, Z). % and so, transitively, to Z
```

The second rule is a **transitive closure**: the set of all entities an entity depends on, directly or through others. The chapter's advice, that future dependency or impact analysis should be an explicit analysis over the model producing derived outputs, is exactly this pattern. The model is never modified, and the result is a new value.

◆ Impact analysis is reverse reachability

The question "if I change Account, what else is affected?" asks for every entity from which Account can be reached by following relationships, that is, the entities that depend on it. That is **reachability in the reversed graph**, computed by breadth-first search in time proportional to the number of entities plus relationships.

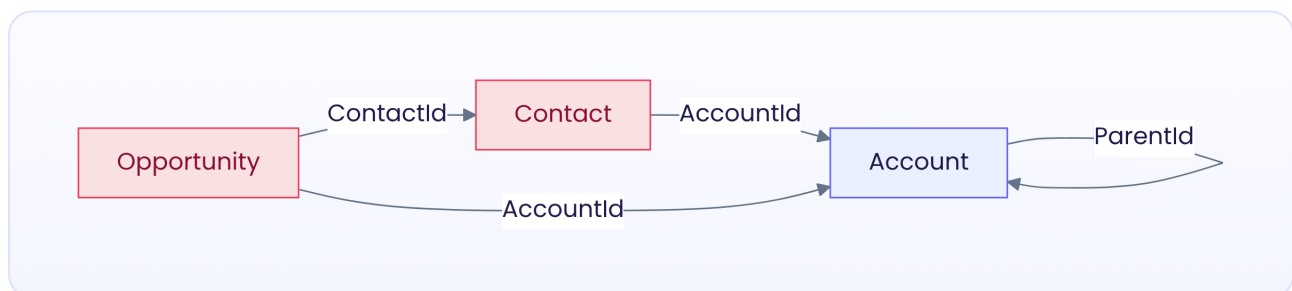


FIGURE 16 Impact analysis is reverse reachability

If Account changes, Contact and Opportunity (shaded) are affected: Contact directly, Opportunity directly and also through Contact. Account's own self-loop adds nothing new.

◆ Load order is a topological order

A frequent practical question for schema work is: *in what order can data be loaded so that every parent exists before its children?* The answer is a **topological order** of the dependency graph, an ordering in which every edge points from a later entity to an earlier one. It exists if and only if the graph has no

cycles. Kahn's algorithm (1962) finds one by repeatedly taking an entity with no unresolved parents:

```
order = [ ]
ready = entities with no outgoing relationships
while ready is not empty:
    take x from ready, append x to order
    for each entity y that refers to x:
        remove that relationship from y's unresolved list
        if y has no unresolved relationships left: add y to ready
if order has fewer entities than the model: the rest lie on or behind a cycle
```

Self-relationships and mutual references create **cycles**, which make a strict order impossible. Real data loads deal with this by two-phase loading: create the records first and fill the cyclic references in a second pass. The algorithm's leftover set tells you exactly where that is needed. Cycles also matter for **drawing**: layered layouts (chapter 12) must break cycles by reversing some edges, a problem known as finding a *feedback arc set*.

◆ Structural measures

Once the model is a graph, standard measures apply and are cheap to compute.

MEASURE	MEANING	WHAT IT REVEALS
In-degree	how many relationships point at an entity	hubs such as Account; changes there are risky
Out-degree	how many relationship fields an entity holds	how dependent an entity is
Connected components	groups of entities linked to each other	separate clusters; candidate diagram pages
Strongly connected components	mutually reachable entities (Tarjan, 1972)	cycles that block a clean load order
Isolated entities	no relationships at all	possible leftovers or typos

ANALYSIS	COST
Degrees	$O(V + E)$
Reachability from one entity	$O(V + E)$
Topological order or cycle test	$O(V + E)$
Strongly connected components	$O(V + E)$
Transitive closure of everything	$O(V \times (V + E))$, or better with tuning

◆ Layout traversal is not source execution

The chapter distinguishes rendering algorithms from execution. Connector routing walks edges, and yet it does not *run* the program. The test is whether the traversal produces meaning or produces a picture. Routing produces pixels, so it is part of geometry. An impact analysis produces information about the schema, so it is a separate, explicit analysis whose output is derived data and never a hidden change to the model.

COMMON PITFALL Analysis that writes back

If an analysis stored its result in the model, for example a "depth" attribute on each entity, every consumer would then see a model that depends on which analyses have run. Return analysis results as separate values keyed by entity identity.

CHECK YOUR UNDERSTANDING

1. For the schema in this chapter, give a valid load order and say which relationship needs a two-phase treatment.
2. Write the reachability computation as a Datalog rule for "affected by Account".
3. What is the in-degree of Account and what does it suggest?

16

Access calculation and analysis

The compiler does not calculate authorization, sharing, or privilege. Structural association must not be interpreted as permission. If access analysis is introduced, it should use typed policy edges or a separate analysis model with documented inheritance and override rules.

A display may combine structural and policy information, but the underlying representations should distinguish their meanings.

◆ Going deeper: structure is not permission

The chapter states a rule with a clear logical basis: *structural association must not be interpreted as permission*. Two different kinds of statement are involved, and mixing them is a **category error**.

	STRUCTURAL GRAPH (THE ER MODEL)	POLICY GRAPH (ACCESS)
A relationship says	that one entity refers to another	that a subject may do something to an object
Absence of an edge means	<i>unknown</i> : the diagram may simply be incomplete	<i>denied</i> : nothing is allowed unless granted
Reasoning style	open world	closed world (deny by default)
Edges are	relationships between entities	grants between roles or profiles and objects
Inheritance rules	none intrinsically	must be defined explicitly

The difference in the second row is the important logical point. A diagram with no line between Account and Invoice does not assert that they are unrelated, only that no relationship was described. A permission model with no grant from a role to an object **does** assert that the role has no access. Treating a missing structural edge as "no relationship" or a present edge as "some permission" reads one kind of statement in the reasoning style of the other.

◆ Typed edges and separate layers

The chapter recommends typed policy edges or a separate analysis model. In graph terms this is a **multi-relational graph** in which the edge type decides how it may be used. Keeping the two graphs as separate layers, joined only by shared entity identities, keeps the meanings clean.

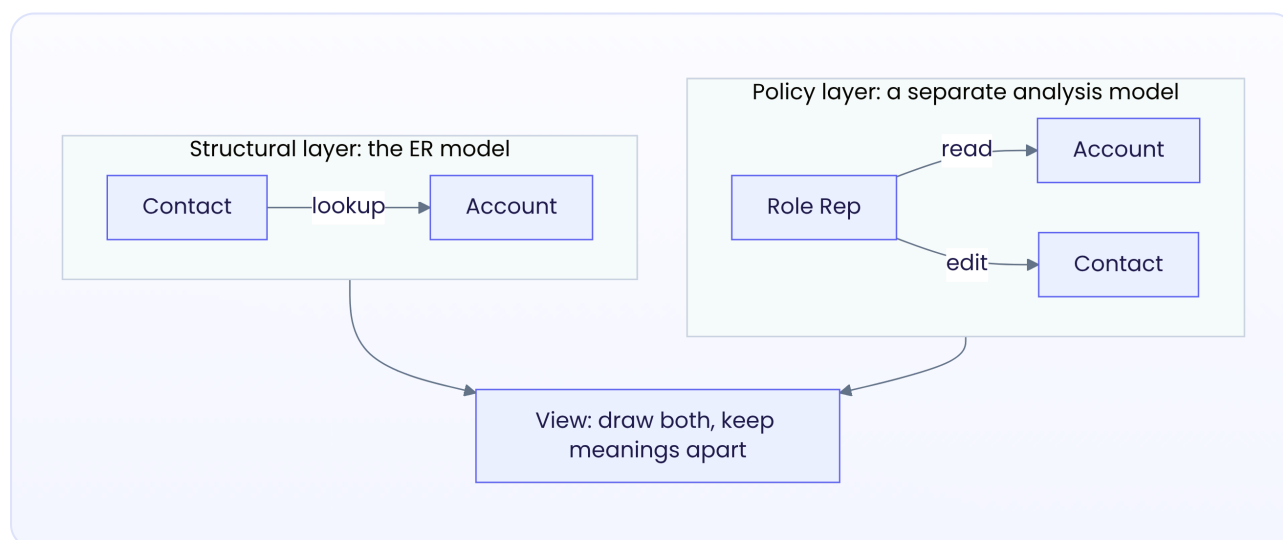


FIGURE 17 Typed edges and separate layers

A **view** may draw the two layers together, for example with dashed lines for structure and coloured badges for policy, as the chapter allows. This is the same technique used in map software: the layers are separate data, and only the rendering combines them.

◆ The platform's own nuance

There is one subtlety that makes this separation *more* important. In Salesforce itself, some relationship kinds do have access consequences: the sharing of a child record whose relationship to its parent is a master-detail relationship is controlled by the parent, whereas an ordinary lookup carries no such consequence. So the relationship **kind** recorded in the model is an input to access reasoning on the platform. The compiler is still right to record the kind as structural data and nothing more. The inheritance rule belongs in the policy layer, which reads the kind from the model and applies the platform's rule. The access calculation in the companion guide on the SML compiler shows what such a policy layer looks like.

FORMAL VIEW Keeping the layers honest

Let S be the structural relation and P the policy relation. Nothing is inferred from one to the other unless a rule says so:

$$S(a, b) \not\Rightarrow P(a, b) \quad \text{and} \quad P(a, b) \not\Rightarrow S(a, b)$$

When the platform does define a rule, such as inheritance along master-detail edges, it is written as an explicit rule R of the policy layer, so that the assumption is visible and testable:

$$P(\text{parent}, x) \wedge S(\text{child}, \text{parent}, \text{masterDetail}) \Rightarrow P(\text{child}, x)$$

◆ What a policy layer needs that a schema does not

NEED	WHY
Documented inheritance and override rules	a grant may be inherited, or overridden by a more specific one
A default (normally deny)	absence must have a defined meaning
Provenance	to explain <i>why</i> an access exists
Separate tests	policy bugs are not schema bugs

COMMON PITFALL The helpful shortcut

"Users can see Contacts, and Contact has a lookup to Account, so users can probably see Accounts" is the kind of inference a diagram invites. It is often wrong, and when it is wrong it misleads a security decision. Never let a rendering suggest access that no policy grants.

CHECK YOUR UNDERSTANDING

1. State in one sentence why an absent edge means different things in the two layers.
2. Which layer should own the rule "master-detail children inherit access from the parent", and what does the compiler contribute?
3. Sketch how you would draw both layers in one diagram without conflating them.

17**Optimization and performance thinking**

There is no conventional optimization pass such as constant folding because declarations do not produce executable instructions. Relevant costs are scanning, model construction, geometry, DOM rendering, and rasterization. Recognition is approximately linear in input size; entity lookup uses a map and field lookup uses arrays appropriate for expected diagrams.

The principal optimizations are visual: edge deduplication, lane fan-out, self-loop routing, and canvas-bound clamping. They must preserve semantic invariants. Measure parser, geometry, browser, and export time separately so bottlenecks are correctly located.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: where the time actually goes

The chapter says there is no conventional optimisation pass, because nothing executable is produced, and names the costs that do exist. To reason about them, give each stage a size parameter. Let n be the number of lines, V the entities, F the total fields, k the largest number of fields in one entity, E the relationships, and P the pixels of the drawing.

STAGE	DOMINANT COST	ORDER OF GROWTH
Recognition	one pass, each line matched by a linear pattern	$O(n)$
Entity lookup	a map by normalised name	$O(1)$ per lookup
Field lookup	a scan of the entity's field array	$O(k)$ per lookup, so $O(F \times k)$ at worst
Deduplication	a set of keys	$O(1)$ per relationship
Geometry	one box per entity, one path per relationship, lane grouping	$O(V + F + E \log E)$
DOM rendering	one element per row and per path	proportional to $V \times \text{rows} + E$
Mermaid layout	a layered layout with crossing reduction	worse than linear, dominated by E and layers
PNG rasterisation	painting every pixel of the image	$O(P)$

Reading the table gives clear guidance. Recognition and model construction are cheap and linear. The first stage where cost can hurt is geometry and rendering, and after that rasterisation, whose cost depends on the image size and not on the source size at all.

◆ Measure each stage separately

The chapter advises measuring parser, geometry, browser and export time separately. Timing the whole action ("time from keystroke to picture") tells you that something is slow and nothing about what. The browser's **User Timing** API allows named marks and measures around each stage, and the performance panel in the developer tools shows them on a timeline.

MEASUREMENT	WHERE TO PLACE IT	WHAT A LARGE VALUE POINTS TO
parse	around <code>parseEr()</code>	pathological input or a slow pattern
geometry	around <code>buildErGeometry()</code>	routing or lane assignment cost
render	around applying the geometry to the DOM	too many elements
export	around each exporter	encoding or XML building cost
rasterise	around the PNG utility	the image is very large

◆ Speed by not doing work

Because the derivations are pure (chapter 3), the cheapest optimisation is to skip recomputation whose inputs are unchanged.

SITUATION	TECHNIQUE	SAFE BECAUSE
The user is still typing	debounce : wait for a short pause before compiling	only the last state matters
The text has not changed	compare with the previous text and reuse the model	<code>parseEr</code> is a function of the text alone
A box is being dragged	recompute geometry only, reusing the model	only an override changed
A large model is shown	level of detail : hidden-field counts, and fan-out lanes	the summary preserves meaning

The chapter's list of visual optimisations (edge deduplication, lane fan-out, self-loop routing and bound clamping) are all this kind of technique. Their shared requirement is the chapter's warning: they must **preserve semantic invariants**. Deduplicating edges for drawing must not remove a relationship from the model, and hiding fields must not remove them from the model either.

◆ Amdahl's law in one example

The benefit of speeding up one stage is limited by how much of the total time it accounts for. If a fraction p of the time is made s times faster, the overall speed-up is $1 / ((1 - p) + p / s)$. Suppose, purely for illustration, that in a large export rasterisation accounts for 60% of the time and everything else for 40%. Making parsing ten times faster changes nothing measurable, because parsing is well under 1% of the total. Halving rasterisation time ($p = 0.6$, $s = 2$) gives a speed-up of $1 / (0.4 + 0.3) \approx 1.43$. The lesson is to spend effort where the measurements say the time is.

◆ Verifying that an optimisation is safe

An optimisation that changes output is a bug. The reliable check is **differential testing**: run the optimised and the straightforward implementation on many inputs and assert that the results are equal. For caches, assert that a cached result equals a freshly computed one.

COMMON PITFALL

Optimising the parser

It is tempting to tune the regular expressions because they are the most visible code. They are also, for realistic inputs, the least significant cost. Profile before optimising, and remember that a pattern that is fast today (chapter 6) is linear whatever its constant.

CHECK YOUR UNDERSTANDING

1. For a model with 500 entities and 2,000 relationships, which stage would you expect to be slowest in the browser, and how would you confirm it?
2. Design a cache key for geometry, listing every input the result depends on.
3. Explain in your own words why hiding fields is an optimisation that must not touch the model.

18**Diagnostics and error recovery**

Failures include unrecognized lines, malformed declarations, and inconsistent input. A useful diagnostic contains severity, line/column, stable code, message, and correction guidance. The current exception-style API is adequate for synchronous compilation; a richer language service could return `{ model, diagnostics }` and continue after errors.

Recovery may skip a malformed line or retain an error node, but must not invent business meaning silently. A partial model with a visible diagnostic is safer than a plausible but incorrect diagram.

GOING DEEPER: THEORY AND LOGIC◆ **Going deeper: error handling in a line-oriented language**

The chapter describes the current failure style as exception-based: adequate for synchronous compilation, with a richer alternative of returning a model together with diagnostics. The difference between the two is a well-known design choice.

	EXCEPTION STYLE (CURRENT)	RESULT STYLE (PROPOSED)
First error	aborts the compilation	is recorded and compilation continues
Output	nothing, or an error	a partial model and a list of diagnostics
Good for	scripts and batch runs	editors that must show every problem at once
Type of the result	a model, or an exception	a pair (<i>model</i> , <i>diagnostics</i>)

◆ **Line orientation makes recovery easy**

Every language processor that meets an error has to decide where to *resume*. In a language with nested blocks, that is a hard problem: the parser skips tokens until it finds something it trusts (this strategy is called **panic mode**), and a wrong guess causes a cascade of follow-on errors. A line-oriented language

has a built-in answer: **the end of the line is a perfect synchronisation point**, because no construct spans lines. To recover from a malformed line, discard the line and continue with the next. No guesswork is involved, and one bad line cannot corrupt the meaning of the others.

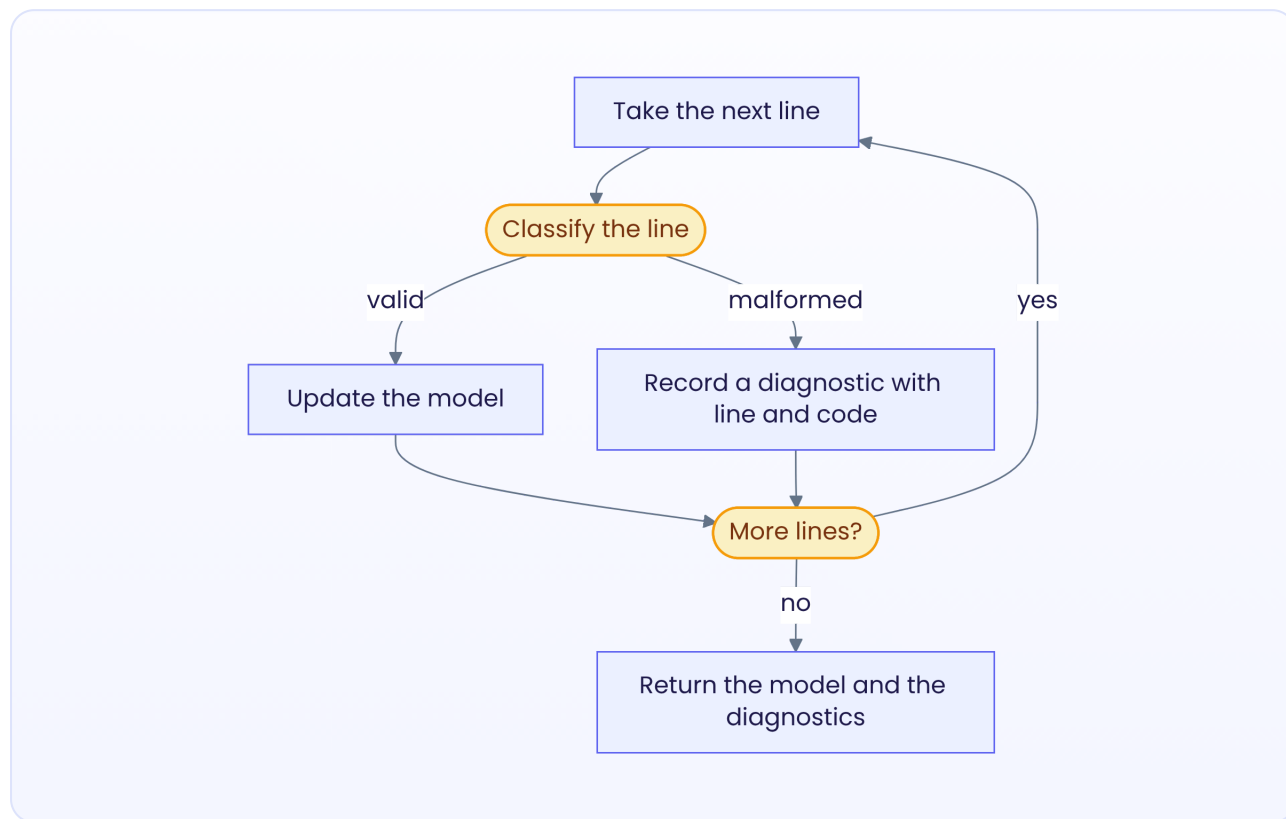


FIGURE 18 Line orientation makes recovery easy

The chapter adds a rule that keeps recovery honest: recovery may skip a line or keep an error node, but it must not **invent business meaning silently**. A partial model with a visible diagnostic is preferable to a plausible but incorrect diagram.

◆ The anatomy of a useful diagnostic

The chapter lists severity, line and column, a stable code, a message and correction guidance. Each field has a purpose.

FIELD	PURPOSE	CONSUMER
Severity	how serious: error, warning, information, hint	editor, export gating
Range (line, start and end column)	where to underline	editor
Code	a stable identifier, so tests and users do not match on wording	tests, documentation
Message	what is wrong, in plain words	the user
Suggestion	how to fix it	the user, quick-fix actions

Column ranges are practical to obtain. Since ES2022, JavaScript regular expressions accept a `d` flag that returns the start and end index of each capture group, so the position of the operator, the child or the parent within a line is available without extra parsing.

◆ A proposed catalogue

The following table is a *proposal* for stable codes, consistent with the failures the chapter mentions. The numbering scheme groups codes by stage.

CODE	MEANING	SEVERITY	SUGGESTION
ER101	line is not recognised	error	expected <code>entity Name</code> or <code>Child.Field -> Parent</code>
ER102	unknown relationship operator, such as <code>--></code>	error	did you mean <code>-></code> , <code>=></code> or <code>~></code> ?
ER103	entity declaration with no name	error	write <code>entity Name : fields</code>
ER201	relationship target was never declared	information (strict mode: error)	declare it, or check the spelling
ER202	duplicate relationship, ignored	information	remove the repeated line
ER203	same field declared with two relationship kinds	warning	choose one operator
ER204	unrecognised bracket text kept as display type	hint	none
ER301	a label was altered to fit the target format	information	none

Suggestions such as "did you mean" rely on edit distance (chapter 10). For a mistyped keyword like `Entiy`, the distance to `entity` is 1, so a fix can be offered safely.

◆ Diagnostics and testing

Stable codes turn diagnostics into testable behaviour: a test can assert that a given source produces exactly the codes `ER102` and `ER201`, and nothing else, with the reported line numbers. Message wording can then change without breaking tests.

COMMON PITFALL Messages that need context

A message such as "invalid syntax" tells the user nothing. A message must be understandable *without* looking at the source, so it should include the offending text and what was expected. Errors are the most-read text in any tool.

CHECK YOUR UNDERSTANDING

1. For the source `entity Account : Name` followed by `Contact.AccountId --> Account`, write the diagnostics a result-style compiler would return.
2. Why is it useful that a bad line does not stop later lines from being processed, and what risk does it introduce?
3. Design the quick-fix action for `ER102`.

19

The code-generation model in the DSL

Each backend targets a different grammar. Mermaid requires identifier-safe type tokens; draw.io requires XML escaping, IDs, and coordinates; SVG consumes geometry; PNG rasterizes SVG separately. Escaping and sanitization therefore belong at target boundaries, not in the parser.

Generators should be deterministic. Stable ordering and IDs improve tests, caching, diffs, and trust. Test special characters, empty entities, all relationship kinds, self-relationships, and parallel edges.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: generating text for four different grammars

The chapter's central point is that each backend targets a different grammar, and so encoding belongs at the target boundary. The table shows what each backend consumes and what it must be careful about.

BACKEND	CONSUMES	ITS OWN GRAMMAR	ENCODING TO GET RIGHT	LAYOUT COMES FROM
Canvas (SVG)	geometry	XML and SVG	escape text; numeric coordinates	the geometry IR
draw.io	geometry	XML, cells with styles and ids	escape attributes; unique ids; coordinates	the geometry IR
Mermaid	model	Mermaid's ER syntax	quote labels; identifier-safe type tokens	Mermaid itself
PNG	SVG text	raster image	correct rasterisation of the SVG	not applicable

◆ Injectivity: sanitising must not merge distinct things

Some target grammars accept only restricted identifiers, so a name has to be **sanitised**. There is a logical requirement on such a mapping. For *identifiers* (things that must stay distinct) the mapping has to be **injective**: two different inputs must never map to the same output. For *labels* (things that are only displayed) the mapping may be lossy, because collisions do no harm.

FORMAL VIEW

When sanitising is safe

Let *sanitise* map names to target-safe tokens. It is safe for identifiers when

$$a \neq b \Rightarrow \text{sanitise}(a) \neq \text{sanitise}(b)$$

The entity names in this DSL are restricted to word characters, so they need no sanitising and this property holds trivially. Field types are free text (for example a type written with parentheses or braces) and only appear as labels or type tokens. Turning `Text(255)` into a safe token such as `Text_255` is a lossy mapping, which is acceptable because it is display-only, and would not be acceptable for something used as a key.

◆ What each encoder must handle

Mermaid. Quote labels so that braces, parentheses, slashes and line breaks are treated as text. In Mermaid's ER syntax a line such as `Account ||..o{ Contact : "AccountId"` describes a relationship; the two dots draw a *non-identifying* (dashed) connection and two hyphens an *identifying* (solid) one, which is why the chapter's example of a lookup produces a dashed line. Mermaid also allows the key markers `PK`, `FK` and `UK` on attributes, which gives relationship fields a natural home.

draw.io. The format is XML. Attribute values must escape ampersands, angle brackets and quotes, every cell needs a unique identifier, and the two structural cells at the root that every file contains must be present. Coordinates must be numbers, and style strings must not be assembled from unescaped text.

SVG. Text nodes need the same XML escaping. A second problem is **text measurement**: to size a box, geometry must know how wide its text will be. If it uses one estimate and the browser draws with a different font, boxes and text disagree. Accepting a measuring function as a parameter (chapter 4) lets tests use a fixed width.

PNG. Rasterising an SVG depends on the browser's rendering, so it is the least deterministic backend. Compare PNGs by dimensions and structure and, where necessary, by a small tolerance, not by exact bytes.

◆ Determinism

The chapter asks generators to be deterministic. Three habits deliver it: iterate over collections in a defined order (the insertion order of a map is defined in JavaScript, and sorting is available where an explicit rule is needed), derive identifiers from content and not from a counter that depends on order (chapter 12), and never put timestamps or random values in output. The benefits listed in the chapter follow directly: a diff between two exports shows only real changes, tests can compare against a stored expected output, and a cache can reuse an output safely.

◆ The DSL as a target

The import direction, turning platform metadata into DSL text, makes the DSL itself a *target language*. Its generator is a **pretty printer**, and a natural correctness property applies: printing a model and parsing the result must give the same model back.

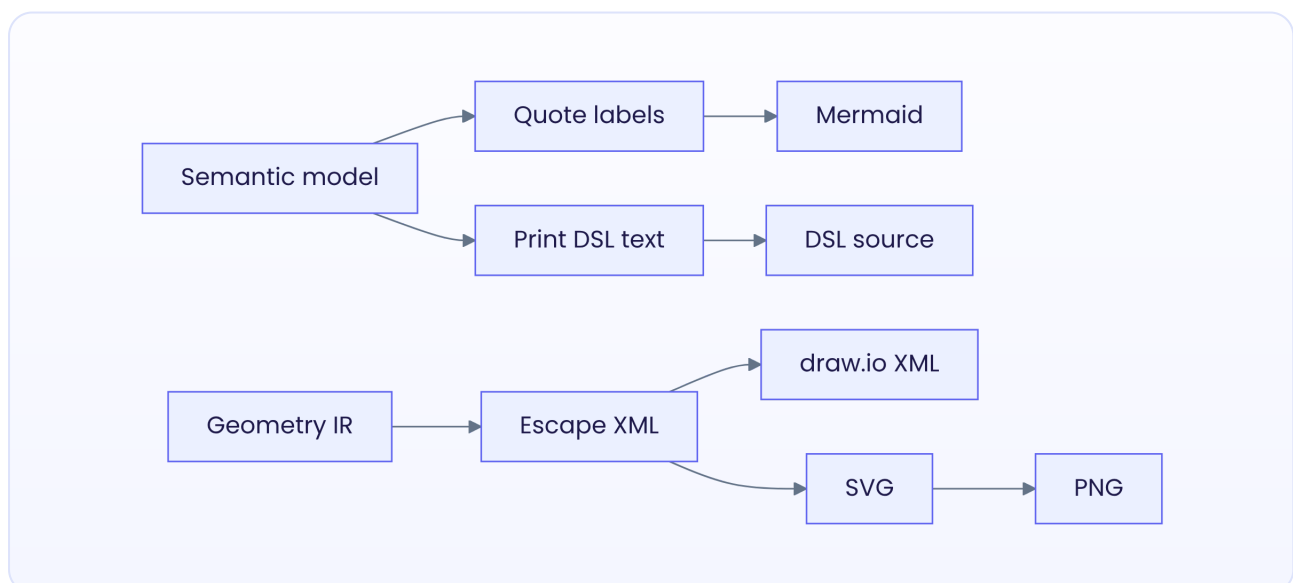
$$\text{parse}(\text{print}(m)) = m$$


FIGURE 19 The DSL as a target

The diagram shows the encoding step as a separate stage in front of every output, which is where the chapter says escaping belongs.

COMMON PITFALL Testing only friendly names

Most bugs in generators appear with unusual input: an entity with no fields, a type with parentheses, a self-relationship, two parallel relationships between the same pair. The chapter lists these; make them fixtures in every backend's test file.

CHECK YOUR UNDERSTANDING

1. Which of the four backends could be unaffected by a name containing an ampersand, and why?
2. Write a property test for the print-then-parse equation. What kinds of models must the generator produce?
3. Why is comparing PNG files byte for byte a fragile test?

20

Editor integration and language services

The semantic model supports entity and field completion, relationship suggestions, arrow completion, lint hints, and import assistance. The editor must reuse compiler normalization rules rather than implement a second parser.

Pure parsing and geometry functions can be debounced and cached. Context-sensitive completion should use the cursor position: after `Contact.` suggest fields, after an operator suggest entities, and inside brackets suggest markers or known types. Diagnostics and rendering must agree about case sensitivity and implicit declarations.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: completion, diagnostics and one source of truth

The chapter says the editor must reuse compiler normalisation rules and not implement a second parser. This is the classic argument for a **single source of truth**: if two pieces of code each encode the same rule, they will eventually disagree. The symptoms are familiar.

SYMPTOM	ROOT CAUSE
The editor underlines a line the compiler accepts	the editor has its own, stricter pattern
Completion offers <code>account</code> while the compiler shows <code>Account</code>	two different case rules
A quick fix creates text the parser then rejects	the fix generator does not share the grammar

The remedy is architectural: the editor calls the compiler's functions (recognition, the identity key, the model) and does not re-implement them. A test that compares the editor's diagnostics with the compiler's for a corpus of sources keeps them aligned.

◆ Completion is matching a prefix

Completion has to answer "what can legally come next?" from *incomplete* text. In formal-language terms the text so far is a **prefix** of some string in the language, and the set of continuations is what remains after consuming that prefix. For regular languages this is a neat, well-defined operation: the *derivative* of a regular expression with respect to a prefix (Brzozowski, 1964) is precisely the regular expression describing all valid continuations.

The practical version does not need derivatives. It classifies the text **before the cursor** with a small set of patterns, each of which recognises a *prefix* of a valid line and names the kind of thing expected next. All of these were checked on sample text.

TEXT BEFORE THE CURSOR	PREFIX PATTERN (SKETCH)	SUGGEST
Contact.	word, dot, optional partial word	fields of Contact
Contact.AccountId ->	word, dot, word, operator, optional partial word	entity names
Contact.AccountId -> Acc	the same, with a typed prefix	entity names starting with Acc
entity Account : Name, Annual	entity , name, colon, text	field names or known types
entity Account : Name[Cur	an unclosed [at the end	markers and known types
an empty line or a partial keyword	start of line	entity , or an entity name

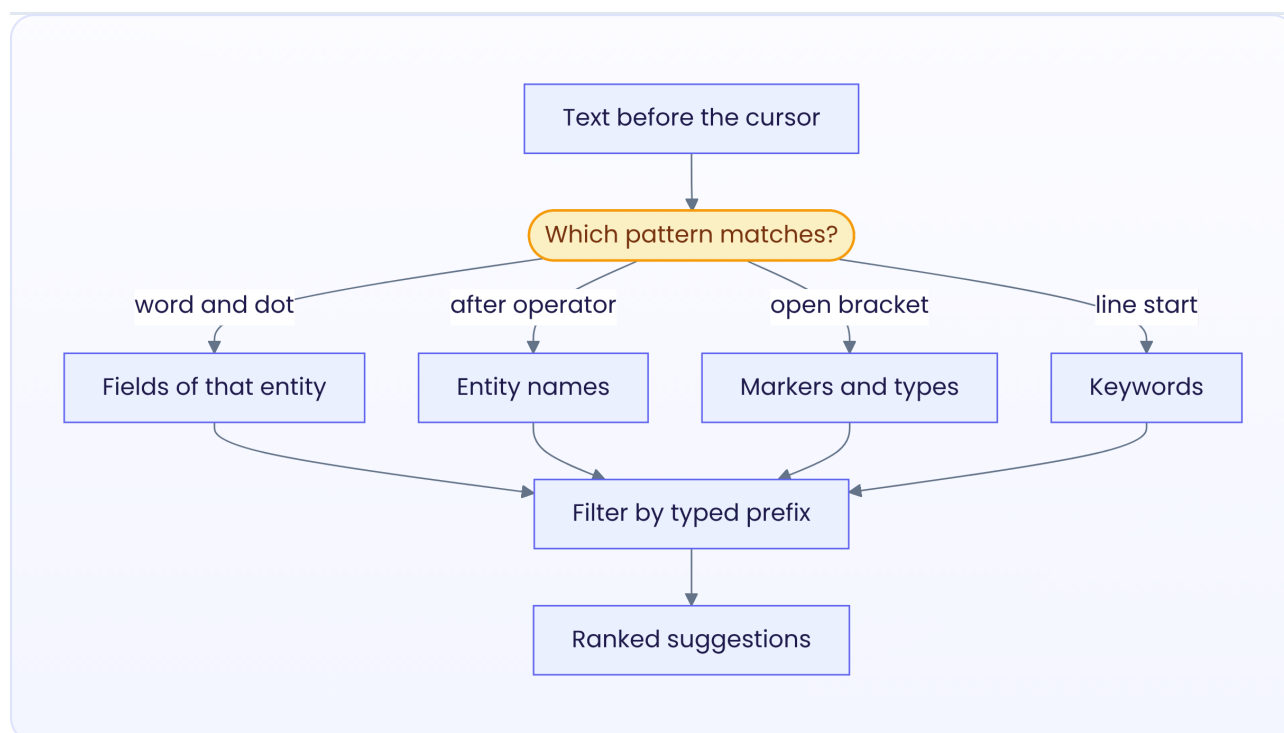


FIGURE 20 Completion is matching a prefix

Two details keep completion consistent with the compiler. Matching on the typed prefix must be **case-insensitive**, using the same identity key as chapter 10, and the suggestion must insert the *preferred display spelling*. A completion that inserts `account` when the entity is displayed as `Account` invites the display-name inconsistency the chapter warns about.

◆ Import assistance and the round trip

The chapter mentions import assistance. Turning Salesforce metadata into DSL text is the *generation* direction (chapter 19), and it interacts with completion: the same registry of entities and fields that feeds the importer feeds the suggestions. A round trip test protects both: import, print, parse, and compare with the metadata that started the process.

◆ Lint hints

Because the model is a graph, several helpful hints cost almost nothing to compute.

HINT	HOW IT IS FOUND
Probable typo in a relationship target	an undeclared entity within a small edit distance of a declared one
Isolated entity	no edges in or out
Unused declared entity	declared but never referenced (may be intentional)
Duplicate relationship	the deduplication step (chapter 11) recorded a repeat

◆ Latency

Interactive editors live within a response-time budget. Research on usability suggests that about a tenth of a second feels instantaneous, and about a second keeps the user's train of thought. The remedies in chapter 17 apply directly: **debounce** compilation while typing, reuse the last model when the text is unchanged, and keep parsing and geometry as pure functions so results can be cached.

COMMON PITFALL Completion from stale state

Suggestions computed from the model of the *previous* compile offer entities the user has just deleted. Compute completions from the most recent successful parse, and mark the model as stale when the text has changed but has not yet been recompiled.

CHECK YOUR UNDERSTANDING

1. Write the prefix pattern for "after the dot" completion and list two lines that it must not match.
2. Why does completion need the identity key, and what would happen if it did not use it?
3. Which lint hint would you enable by default, and which would you make optional?

21 Testing the compiler properly

Tests should be layered: recognition, semantic normalization, geometry, backend emission, and end-to-end rendering/export. Semantic tests have the highest leverage because every target depends on the model.

Property-based tests can enforce invariants such as one entity per normalized identity, no exact duplicate edges, positive box dimensions, and non-degenerate self-loops. Golden files are useful for complete output but should be combined with structural assertions to avoid brittle formatting tests.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: invariants, laws and input-space partitioning

The chapter asks for layered tests and singles out semantic tests as the highest-leverage layer, because every target depends on the model. It also recommends property-based tests for invariants. This section turns that advice into a concrete plan.

LAYER	QUESTION	NEEDS A BROWSER?
Recognition	does each line classify correctly?	no
Semantic normalisation	is the model correct?	no
Geometry	are sizes and paths valid?	no
Backend emission	is each output well-formed?	no
End to end	does the whole export work?	yes

◆ Properties of the ER DSL compiler

A **property** is a statement that must hold for *every* input in a class, which a generator tries to falsify with many random inputs. The invariants named in the chapter and the laws from chapter 11 give a ready list.

PROPERTY	STATEMENT	BUG IT CATCHES
One entity per identity	no two entities have equivalent names	broken case normalisation
No duplicate edges	no two relationships share a normalised key	duplicate detection missing
Idempotence	repeating any line changes nothing	state leaking between lines
Order independence	permuting lines gives the same entities, fields and edges	a hidden ordering assumption
Case independence	changing the case of a reference does not change the structure	identity compared by display name
Comment neutrality	adding blank and comment lines changes nothing	a classifier that trips on layout
Determinism	parsing the same text twice gives equal results	hidden state, random ids
Totality	parsing any string terminates without an unexpected exception	patterns that hang or crash
Positive dimensions	every box has width and height above zero	zero-height boxes for empty entities
Non-degenerate self-loops	every loop has a positive extent	invisible loops
Edge conservation	the number of edges in each backend equals the model's	a backend that drops parallel edges
Round trip	print then parse gives the same model	a printer or parser drift

◆ Metamorphic relations

When no oracle gives the exact expected output, you can still test that changing the input *in a known way* changes the output in a known way. These are **metamorphic relations**, and they suit a declarative language because so many transformations are meaning-preserving.

CHANGE THE INPUT BY	EXPECT THE MODEL TO BE	EXCEPT
inserting blank or comment lines	identical	nothing
shuffling the order of lines	identical as a set of entities, fields and edges	display spelling and drawing order
duplicating a relationship line	identical	nothing
changing the case of a reference	identical structure	display spelling
adding one relationship line	the old model plus one edge, and possibly one entity	nothing

◆ Partitioning the input space

Exhaustive testing is impossible, so testers divide the inputs into **equivalence classes**, groups of inputs the program should treat alike, and test one or two from each class plus the **boundaries** between classes. For this DSL the classes come from the chapter's own list.

DIMENSION	CLASSES AND BOUNDARY VALUES
Line kind	blank, comment, entity, relationship, malformed
Fields per entity	zero, one, many, exactly the visible limit, one more than the limit
Relationship operator	<code>-></code> , <code>=></code> , <code>~></code> , an invalid operator
Reference order	target declared before, after, or never
Repetition	none, exact duplicate, same field with a different kind
Shape	ordinary, self-relationship, parallel edges
Characters in labels	plain, parentheses, braces, quotes, ampersand

The boundary rows are worth attention: an entity with *exactly* as many fields as the box can show, and one with one more, is where the hidden-field logic changes behaviour. **Pairwise (all-pairs) testing** reduces the combinations by requiring only that every *pair* of values from two dimensions appears together in some test, which catches most interaction bugs with far fewer cases than the full product.

◆ Golden files and structural assertions

The chapter warns that golden files alone are brittle. A whitespace change in an exporter breaks every stored output. Pair them with **structural assertions**: parse the exported Mermaid with the real Mermaid parser, and the draw.io output with an XML parser, and assert facts about the result (the number of entities and edges, that every label survives). Successful parsing is a weak oracle, but it catches exactly the injection and escaping bugs of chapter 3.

◆ Fuzzing and hostile input

Two more techniques earn their place. **Fuzzing** feeds random or randomly mutated text to the compiler and watches for crashes or hangs, and a fuzzer that generates lines from the grammar is particularly effective. A **regular-expression stress test** feeds very long near-matching lines and checks that time grows linearly, a guard against the slowdowns described in chapter 6. **Mutation testing** deliberately changes the code (for instance replacing a lower-case with an identity function) and checks that some test fails, which shows whether the tests really constrain behaviour.

COMMON PITFALL Testing what is easy to write

Tests written by looking at the code tend to check the cases the author already thought about. The equivalence-class table is a deliberate counterweight: it is derived from the *input space*, not from the implementation.

CHECK YOUR UNDERSTANDING

1. Write the order-independence property as a test: what does the generator produce and what is compared?
2. Choose the boundary values for the hidden-field logic if the box shows at most six rows.
3. Why does a successful parse of the Mermaid output not prove that the diagram is correct, and what does it prove?

22

Complete example end-to-end

```
entity Account : Name, AnnualRevenue[Currency]
Contact.AccountId -> Account
```

The first line creates `Account`, `Name`, and `AnnualRevenue` with type metadata. The second creates `Contact`, marks `AccountId` as a relationship field, resolves `Account`, and records a lookup edge. Geometry creates two boxes and a connector; Mermaid produces a dashed non-identifying

relationship; draw.io emits positioned cells; SVG renders the live view; PNG rasterizes SVG. No backend reparses the original text.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: tracing two lines all the way down

The example in this chapter has only two lines, which makes it ideal for following a program through every stage. The trace below fills in what the chapter describes, and marks clearly where a detail is illustrative and not taken from the source.

```
entity Account : Name, AnnualRevenue[Currency]
Contact.AccountId -> Account
```

◆ Stage 1: classification and capture

Each line is tested in dispatch order. Running the two patterns on the sample lines gives the captured pieces.

LINE	MATCHED AS	CAPTURED PIECES
entity Account : Name, AnnualRevenue[Currency]	entity	name Account ; field text Name, AnnualRevenue[Currency]
Contact.AccountId -> Account	relationship	child Contact ; field AccountId ; operator -> ; parent Account

◆ Stage 2: the semantic model

The field text is split (respecting the brackets) into Name and AnnualRevenue[Currency] ; the bracket text Currency becomes the display type. The second line implicitly creates Contact , creates its field AccountId marked as a relationship, resolves Account through the identity map (no new entity), and records one edge.

FORMAL VIEW The resulting model

```
V = { Account, Contact }
fields (Account) = { Name, AnnualRevenue (dataType Currency) }
fields (Contact) = { AccountId (isRelationship, relatesTo Account) }
E = { ( Contact, AccountId, Account, lookup ) }
```

Every invariant of chapter 13 can be checked by hand: both edge endpoints are entities, names are unique, there is one edge, the relationship field is marked, and each field has all its defaults.

◆ Stages 3 to 6: derived artefacts

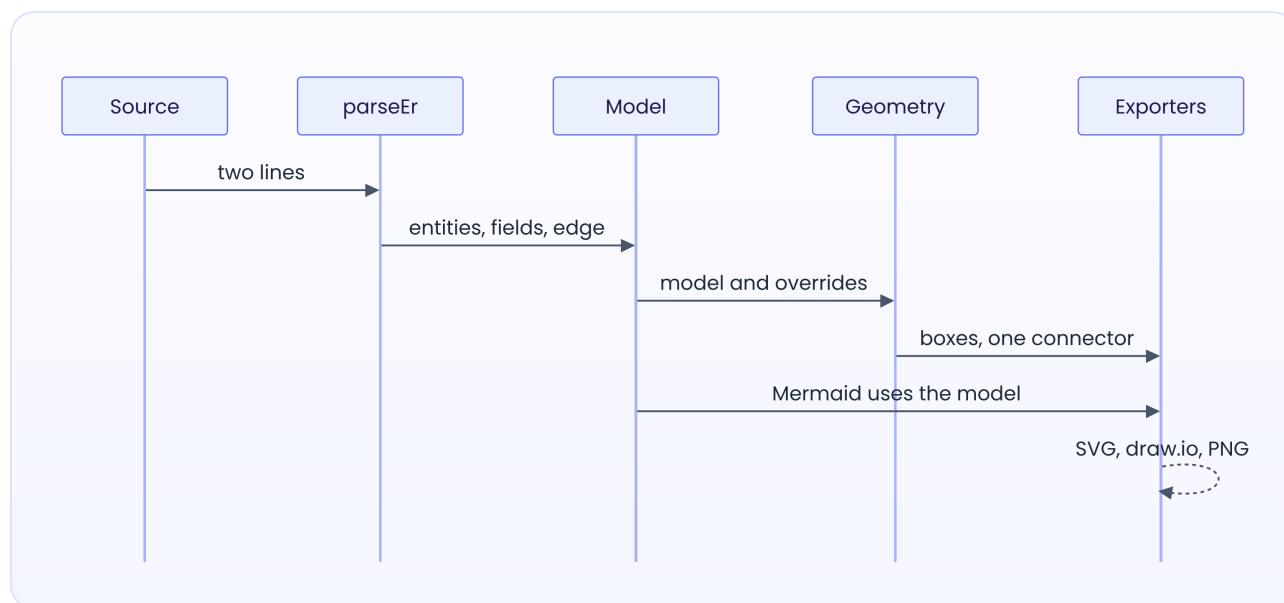


FIGURE 21 Stages 3 to 6: derived artefacts

ARTEFACT	WHAT IT CONTAINS FOR THIS EXAMPLE	NOTE
Geometry IR	two boxes (Account with two rows, Contact with one) and one connector between them	sizes and positions are illustrative and depend on the layout rules
Mermaid text	an entity block for each entity and one relationship line with a dashed, non-identifying connector	exact text depends on <code>buildMermaidErDiagram()</code>
draw.io XML	two vertex cells and one edge cell with positions	ids and coordinates are generated
SVG and PNG	the drawing of the geometry, then its rasterisation	no stage reparses the source text

The last row states the essential invariant of the whole architecture: after the model exists, **nothing looks at the text again**.

◆ A tour of edge cases

The most instructive way to test a mental model is to feed it awkward input. The table gives what happens under the patterns and policies described in this guide, and marks the cases where the guide is silent and a policy should be chosen.

INPUT	WHAT HAPPENS	POLICY STATUS
<code>Contact.AccountId -> Account</code> before any <code>entity</code> line	<code>Contact</code> and <code>Account</code> are created implicitly	documented (forward references)
The same relationship line twice	one edge; the duplicate is ignored	documented (deduplication)

INPUT	WHAT HAPPENS	POLICY STATUS
<code>contact.accountid -> account</code> after the original	the same entity and field; should be the same edge	depends on key normalisation, test it
<code>entity Account</code> with no colon	an entity with no fields	accepted by the pattern
<code>Contact.AccountId --> Account</code>	rejected as an unrecognised line	documented (unknown syntax)
<code>Contact.AccountId -> Account # note</code>	rejected, because the pattern is anchored	decide and document
<code>entity Account : Name # note</code>	accepted, with the comment as field text	decide and document
<code>entity 1Account</code>	accepted although not a valid platform name	domain validation gap
<code>Account.ParentId -> Account</code>	a self-loop	supported, must be non-degenerate

WORKED EXAMPLE Reading the trace as a checklist

Follow the same two lines through your own build: print the captured groups, the model, the geometry and each export, and confirm at each stage the properties listed for that stage. A bug will show up as the first stage whose output disagrees with the table.

CHECK YOUR UNDERSTANDING

1. Extend the example with `Opportunity.AccountId -> Account` and redraw the model, the graph and the invariants.
2. Where in the trace would a misspelt `Account` become visible, and what would the model look like?
3. Write the expected Mermaid relationship line for a self-relationship using the syntax described in chapter 19.

23

Extending the language

Use this sequence: define syntax and ambiguity rules; update recognition; normalize into explicit model fields; update defaults and consumers; update geometry; update every backend; add positive, negative, semantic, and export tests; document compatibility.

A new marker is incomplete if only `parseEr()` recognizes it. It must survive field construction, geometry rows, Mermaid output, draw.io labels, and tests. Extend from the model outward, not from one renderer inward.

◆ Going deeper: extending from the model outward

The chapter's advice is "extend from the model outward, not from one renderer inward". There is a structural reason. Every consumer of the model depends on it, and nothing depends on a renderer, so a change begun at the model reaches every consumer in a controlled way, while a change begun at a renderer reaches only that renderer.

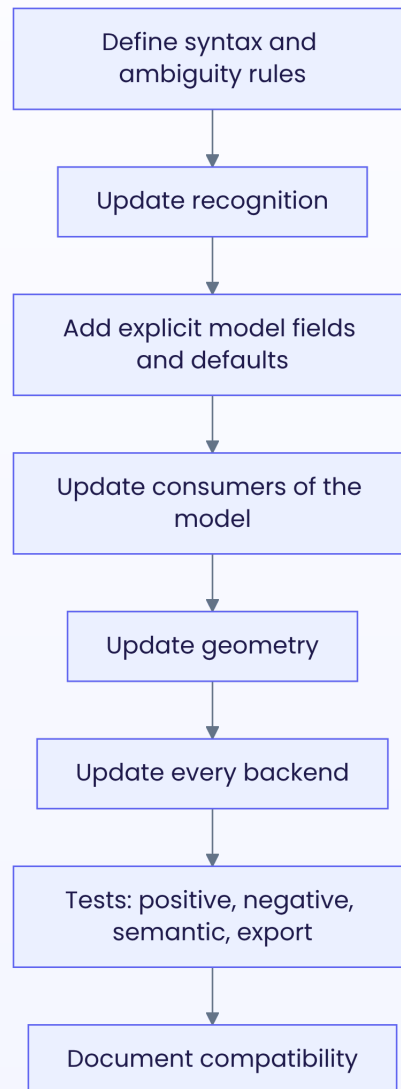


FIGURE 22 Going deeper: extending from the model outward

◆ The expression problem in miniature

The situation is a small instance of the **expression problem**: a system has a set of *kinds* of thing (relationship kinds, field markers) and a set of *operations* over them (recognise, normalise, lay out, and each backend's emit). When each operation handles every kind with its own branch, adding a new kind

means visiting *every* operation. The chapter's warning that a marker recognised only by `parseEr()` is incomplete describes this precisely.

A NEW MARKER MUST APPEAR IN	WHY
the grammar and recogniser	so the source can express it
the model, with a default	so every field has it and consumers need no checks
geometry rows	so it can be shown
Mermaid	so it survives export, or is deliberately dropped
draw.io labels	the same
tests, in every layer	so removal is noticed
documentation	so users know it exists

◆ A worked extension

Suppose, hypothetically, the language gained a marker `Unique`. Mermaid's ER syntax offers an attribute key `UK` for exactly this idea, so the concept has a natural target. Following the chapter's sequence: add `Unique` to the marker vocabulary and grammar; recognise it in the bracket text; add `isUnique` to the field with the default `false` in `finalise`; show it in geometry rows if desired; emit `UK` in Mermaid and a label in draw.io; add tests for the marker alone, combined with other markers, and in each backend; document it.

◆ The compatibility trap

This example hides a real difficulty. The chapter says unknown bracket content is **retained as display text**. That means existing sources may already contain `[Unique]` as free display text. Making it a reserved marker *changes the meaning of those programs*: the text that used to be shown as a type is now consumed as a marker. An extension that keeps every existing program meaning the same is called **conservative**, and this one is not.

KIND OF CHANGE	CONSERVATIVE?	MITIGATION
A new line kind with a new keyword	yes, if the keyword was previously a syntax error	none needed
A new relationship operator	yes, if the text was an error before	none needed
A new reserved marker that was previously display text	no	warn for one release, then reserve; or require a different spelling
Changing a default	no	version the schema
Making a permissive case an error (strict mode)	no	offer strict mode as an opt-in setting

Four practical protections are available. Introduce the new word as a **warning first** (chapter 18). Add an explicit **language or schema version** to saved sources. Keep a **regression corpus** of real historical sources with their expected models, and run it on every change so that any difference is a decision. And provide a migration path for users whose sources need to change.

COMMON PITFALL Extending only the parser

A marker that the parser accepts but the exporters ignore is worse than one that is rejected, because it looks supported. The output of one backend will show it and another will not, and users will report the inconsistency as a bug in the backend.

CHECK YOUR UNDERSTANDING

1. Produce the full checklist for adding a relationship kind, in the order of the flow chart.
2. Is adding a new keyword `alias` conservative? What if `alias` could already be an entity name?
3. Design a warning message for the `[Unique]` transition.

24

Current limitations and future improvements

The language is flat, lacks nested expressions and a closed type system, has no conventional optimization pass, and provides limited recovery. Types are primarily display metadata.

Potential improvements include a token stream and AST, structured source-range diagnostics, strict/permissive modes, a Salesforce type registry, schema versioning, stable IDs, incremental parsing, configurable layout, accessibility metadata, and cross-backend conformance tests. Each should be evaluated against complexity, compatibility, and user value.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: what each improvement buys, and what it costs

The chapter lists the current limitations and a set of future improvements, each to be judged against complexity, compatibility and user value. Here the list is examined item by item, so that the order of work is clear.

IMPROVEMENT	WHAT IT BUYS	WHAT IT COSTS	DEPENDS ON
Structured source-range diagnostics	precise underlines, testable codes	capture-group indices, a diagnostics API	the result-style API
Strict and permissive modes	catches typos when wanted, permissive for imports	a mode setting and both behaviours tested	the diagnostics work

IMPROVEMENT	WHAT IT BUYS	WHAT IT COSTS	DEPENDS ON
Salesforce type registry	validation and completion of types	a registry to maintain, version differences	permissive mode
Schema versioning	safe evolution	a version marker, migrations	none
Stable identifiers	durable layouts and clean diffs	an id scheme	identity rules
Token stream and AST	room for nesting and formatting	a larger front end	only if the grammar grows
Incremental parsing	fast feedback on big sources	a cache and invalidation	pure functions
Configurable layout	user control	more parameters to keep deterministic	geometry contracts
Accessibility metadata	usable diagrams for everyone	text generation from the model	the semantic model
Cross-backend conformance tests	consistency between targets	a shared test suite	edge and entity invariants

◆ Incremental parsing follows from independence

Chapter 11 showed that each line contributes to the model independently. That makes incremental work unusually easy, much easier than in languages with nesting. Cache the contribution of every line keyed by its **text**; when the user edits one line, only that line is re-recognised, and the model is the join of the cached contributions.

FORMAL VIEW

Incremental recompilation

$$\text{model} = \sqcup \{ c(\ell) : \ell \in \text{lines} \} \quad \text{cache} : \text{text of } \ell \mapsto c(\ell)$$

After an edit that changes a set of lines Δ , only $c(l)$ for the lines in Δ is recomputed. The join must still be taken over all lines, which costs $O(n)$ but is very cheap compared with recognition. Where even that matters, a persistent data structure can share the unchanged parts of the previous model.

The *drawing* side has a different concern. When the model changes slightly, the picture should change slightly. This is called preserving the **mental map** of the user (Misue and colleagues, 1995): re-running a layout from scratch can move every box, which destroys the user's sense of where things are. Saved positions keyed by stable identity (chapter 2) are the main defence, and the layout algorithm should honour them.

◆ A closed type vocabulary

A registry of Salesforce field types would turn display text into a checked vocabulary: `Currency`, `Text`, `Lookup` and so on. Because the platform adds types over time, the registry should be *versioned* and consulted for **warnings**, not for rejection (chapter 13). Types would also enable better completion (chapter 20) and richer geometry (for example, an icon per type).

◆ Accessible descriptions from the model

A diagram is a picture, and a picture is not usable by everyone. Because the model is semantic, the compiler can generate a **text description**: "Account has two fields. Contact refers to Account through AccountId (a lookup)." This is a small instance of *natural language generation* from a structure. It can be attached to the SVG as a title and description, and exposed to assistive technology. It is a good example of a benefit that only exists because the model, not the drawing, is authoritative.

◆ Prioritising

A simple way to order the list is to plot each item by *value to users* and *cost and risk*, then take high-value, low-cost items first.

PRIORITY	ITEMS
First	structured diagnostics, strict and permissive modes, schema versioning
Next	stable identifiers, type registry, accessibility metadata, conformance tests
Only if the language grows	token stream and AST
When performance requires	incremental parsing, configurable layout

COMMON PITFALL Building the big machinery early

A token stream and AST are attractive because they look like "proper compiler design". For a flat, line-oriented language they add code and tests and prevent nothing. The chapter's own criterion is the right one: add them when nesting, precedence, blocks or multiline constructs appear.

CHECK YOUR UNDERSTANDING

1. Which improvement should come before the type registry, and why?
2. Explain why incremental parsing is easier here than for a language with nested blocks.
3. Write the alternative text description for a model with three entities and two relationships.

25

Final architecture summary

The ER DSL compiler provides line-oriented recognition, shallow dispatch, case-insensitive symbol resolution, incremental semantic normalization, explicit entity/field/relationship structures, optional geometry IR, target-specific emission, diagnostics, language-service support, and layered tests.

Its essential invariant is that the semantic model is authoritative. Geometry, SVG, Mermaid, draw.io, and PNG are derived artifacts, allowing renderers to evolve independently without duplicating language semantics.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: the architecture as a set of invariants

The summary says the essential invariant is that the semantic model is authoritative. It is worth stating what this means as a rule that can be checked, and drawing the whole system with its dependencies pointing the right way.

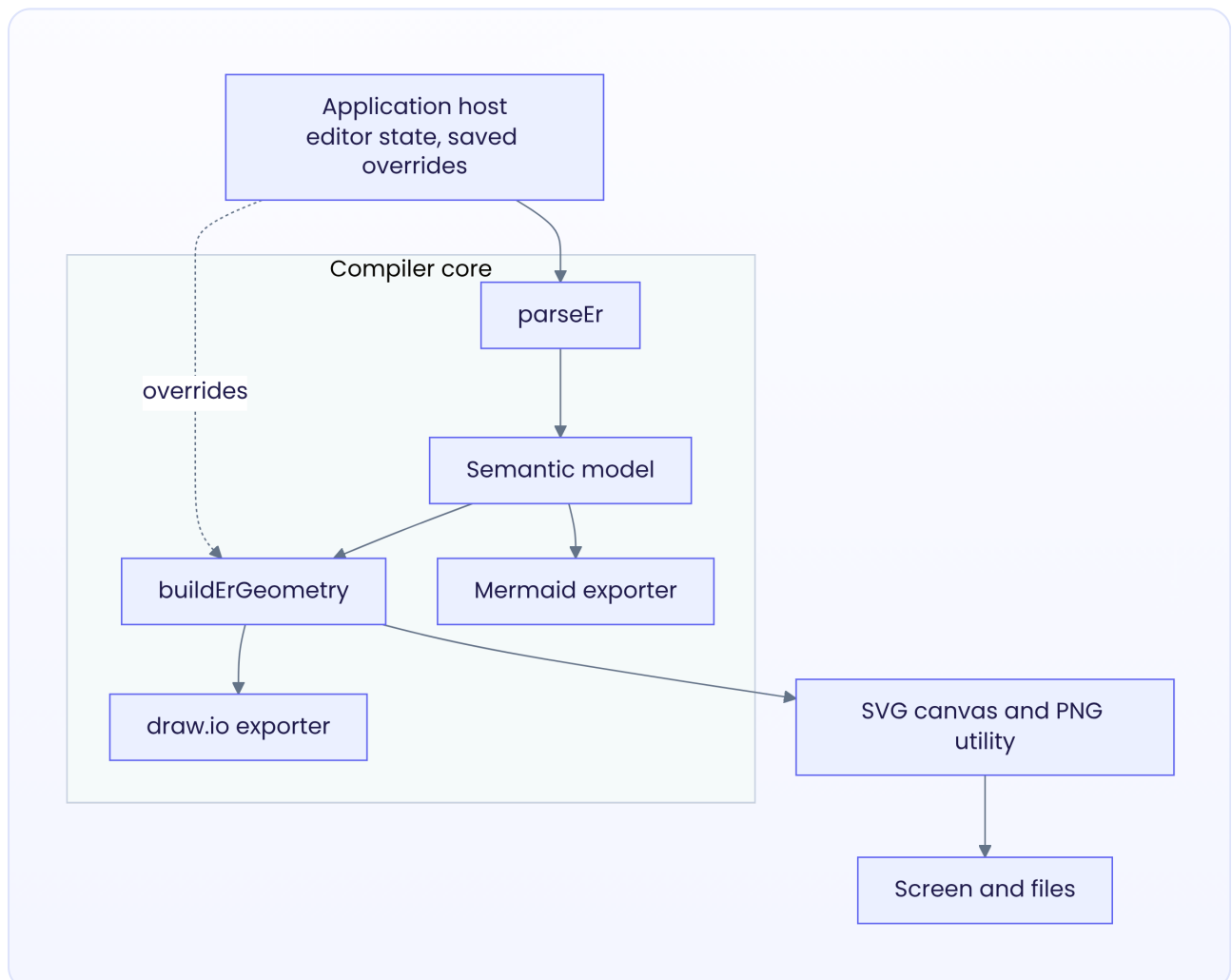


FIGURE 23 Going deeper: the architecture as a set of invariants

FORMAL VIEW**The authority invariant**

Every target is a function of the model (and of layout overrides for the visual ones), and none of them reads the source text:

$$\text{target_t}(P) = \text{f_t}(\llbracket P \rrbracket, \text{overrides})$$

This is a statement about *dependencies*, so it can be enforced mechanically: no exporter or geometry function may import the parser, and no function outside the parser may receive the raw source string.

◆ Fitness functions

Neal Ford and colleagues use the term **fitness function** for an automated check that an architectural property still holds. Invariants like the one above are good candidates, because they otherwise decay silently as code changes. Examples for this compiler:

PROPERTY	AUTOMATED CHECK
Exporters do not read source text	a dependency check that fails if an exporter imports the parser
The core does not touch the browser	a lint rule forbidding DOM globals in the core module
The model has stable defaults	a test that every field of any parsed model has all documented properties
Generators are deterministic	a test that two runs on the same input give identical output
Targets agree on counts	a conformance test comparing entity and edge counts across backends

◆ Quality attributes

ATTRIBUTE	HOW THE DESIGN DELIVERS IT
Fidelity	one model, so all targets agree about meaning
Testability	pure functions, layered tests, no browser needed for semantics
Determinism	defined ordering, stable identifiers, no timestamps in output
Extensibility	extend from the model outward, with a documented checklist
Safety	source is untrusted input, encoded at every output boundary, no code evaluation
Performance	linear recognition, memoisable derivations, level of detail
Usability	tolerant identity rules, forward references, planned diagnostics

◆ Trust boundaries

Source text is **untrusted input**: it is typed by users, pasted from documents and eventually imported. The compiler has no execution semantics, which removes the largest class of risks, but the output boundaries remain: text that reaches an XML file, an SVG image or a Mermaid definition must be encoded for that context (chapter 3). The boundary that matters is between *data* and *syntax of the target*, and it lies at every exporter.

◆ Decisions visible in the design

Writing key decisions down in a short standard form, a **decision record**, keeps the reasoning alive. The table reads the decisions directly off this guide.

DECISION	ALTERNATIVE	CONSEQUENCE
A flat, line-oriented grammar	a block or expression syntax	regular expressions suffice; tree machinery is deferred
Recognition, dispatch and normalisation in one pass	a token stream and AST	less code; harder to add nesting later
Semantic model authoritative	a renderer-centred design	targets cannot disagree about meaning
Two IRs (semantic and geometry)	one combined structure	failures are localised; extra derivation step
Mermaid bypasses geometry	force Mermaid through geometry	Mermaid does its own layout; geometry stays for canvas and draw.io
Case-insensitive identity, first-seen spelling	case-sensitive, or declared-wins spelling	friendly matching; display depends on line order
Implicit creation of relationship targets	strict declaration	forward references work; typos create phantom entities
Permissive metadata preservation	a closed type vocabulary	imports keep working; typos in types are silent
Exception-style failure	a result with diagnostics	simple now; no partial results yet

COMMON PITFALL Decisions without records

A decision that is not recorded is re-argued every year by whoever did not make it. The costs listed in the last column are as important as the choices, because they tell a future maintainer what to watch.

CHECK YOUR UNDERSTANDING

1. Write the fitness function that would have caught a renderer reading the source text.
2. Which quality attribute suffers most from the exception-style API, and how would you improve it?
3. Add two decision records for choices in a project of your own.

26

Design principles every compiler author can reuse

1. Match architecture to grammar.
2. Keep semantics authoritative.
3. Separate syntax from semantic decisions.
4. Make normalization policies explicit.
5. Use stable intermediate representations.
6. Design each backend around its target grammar.
7. Treat diagnostics as part of the language interface.
8. Test invariants as well as examples.
9. Extend from the model outward.
10. Prefer intentional simplicity over ceremonial complexity.

GOING DEEPER: THEORY AND LOGIC

◆ Going deeper: the theory behind each principle

The ten principles are practical rules. Each rests on a body of theory or accumulated experience, and knowing the source helps you decide when a principle applies and when it should bend.

PRINCIPLE	THEORY OR LITERATURE BEHIND IT
1. Match architecture to grammar	the Chomsky hierarchy; the rule of least power; proportionality
2. Keep semantics authoritative	canonical data models; single source of truth; derived data
3. Separate syntax from semantic decisions	static semantics; the limits of context-free grammars
4. Make normalisation policies explicit	equivalence relations and canonical forms; robustness principle
5. Use stable intermediate representations	levels of IR; information hiding (Parnas, 1972); design by contract
6. Design each backend around its target grammar	output encoding; injectivity; pretty-printing
7. Treat diagnostics as part of the interface	error recovery; structured diagnostics; human factors
8. Test invariants as well as examples	property-based testing (QuickCheck, 2000); metamorphic testing
9. Extend from the model outward	the expression problem; conservative extension
10. Prefer intentional simplicity	essential versus accidental complexity (Brooks, 1986)

◆ Proportion, made concrete

The tenth principle is the umbrella for the whole guide. Frederick Brooks separated the difficulty of software into **essential** complexity, inherent in the problem, and **accidental** complexity, added by our tools and methods. An ER schema genuinely has entities, fields and relationships: that is essential. A tokeniser, an AST and a parser generator for a language of two flat line kinds would be accidental. Proportionality is the discipline of adding machinery only when it removes more difficulty than it creates, and the decision table in chapter 1 is the tool for judging that.

◆ Tensions inside this design

None of the principles is free, and several pull against each other. Recognising the tension is part of using the principle well.

TENSION	ONE SIDE	THE OTHER SIDE
Permissive versus strict	imports keep working; unknown input is kept	typos are caught; input is checked
Implicit versus explicit entities	forward references and convenience	phantom entities from typos
One core module versus several	everything near the model, easy to test	several change axes in one file
Regular expressions versus a tokeniser	small and fast now	limited headroom for growth
First-seen versus declaration-wins spelling	simple to implement	order-independent display
Exceptions versus diagnostics	trivial control flow	partial results and richer editors

◆ A checklist for a small declarative language

The following ten questions apply to any tiny DSL, including the next one you write.

1. What are the facts a program can state, and what is the meaning of their union?
2. Is the grammar regular? If yes, write it down and use one pattern per line kind. If no, which construct forces a stack?
3. What is the identity of a thing, and what normal form defines it?
4. What happens on an undeclared reference: create, warn or fail?
5. What is the model, and what invariants must it always satisfy?
6. Which outputs are needed, and what does each output's grammar require of an encoder?
7. What is the display policy when several spellings denote one identity?
8. How are errors reported, and can compilation continue after one?
9. Which behaviours are permissive by design, and where are they documented and tested?
10. What would make you add the machinery you omitted, and who will notice?

CHECK YOUR UNDERSTANDING

1. Pick two principles that pull in opposite directions in your project and describe how you would decide.
2. Answer the ten-question checklist for a small language of your own choosing.
3. For principle 4, list the normalisation policies in this DSL that are currently implicit.

27**Closing thought**

Compiler engineering applies to diagram languages as much as to general-purpose languages. Symbol tables, intermediate representations, diagnostics, code generation, and semantic invariants remain valuable even when the language contains only declarations. The scale of the language changes the amount of machinery required; it does not remove the need for architectural reasoning.

GOING DEEPER: THEORY AND LOGIC◆ **Going deeper: the whole guide in one page**

The closing chapter says compiler engineering applies to diagram languages as much as to general-purpose ones, and that scale changes the amount of machinery but not the need for architectural reasoning. This section gathers the theory of the guide into one view.

STAGE	THE ONE IDEA TO REMEMBER
Philosophy	a declarative program denotes a graph; the model is a canonical hub
Reuse	contracts and three kinds of information keep the layers apart
Pipeline	derivations form a graph of pure functions; encode at every output
Recognition	each line kind is a regular language; one anchored pattern suffices
Structure	trees are for nesting; without recursion there is nothing to record
Grammar	ordered choice over disjoint line kinds; the grammar is regular
Names	identity is an equivalence class; interning gives stable objects
State	parsing is a fold over independent facts; merges should form a semilattice
Representation	a high-level model and a mid-level geometry, with graph drawing for layout
Validation	levels of validity; open world by design, closed world by option
Analyses	graph algorithms over the model, kept separate from policy
Diagnostics	the newline is a perfect recovery point; codes make errors testable

STAGE	THE ONE IDEA TO REMEMBER
Generation	encoders must be correct, injective for identifiers, and deterministic
Testing	invariants, metamorphic relations and partitioned inputs
Evolution	extend from the model outward, and stay conservative

◆ The ladder of machinery

A compiler is not a yes-or-no idea. The amount of machinery grows with the language, and a good design sits on the lowest rung that suffices.

RUNG	ADDS	LANGUAGE IT FITS	WHERE THE ER DSL STANDS
1	regular expressions per line, a model	flat declarations	here
2	a tokeniser and a parser, an AST	nesting, blocks, expressions	not needed
3	a type system and a symbol table hierarchy	typed languages with scope	partly: one global namespace
4	an interpreter or a full IR with control flow	languages that execute	not needed

The companion guide on the SML compiler sits higher on this ladder, with a tree builder, a recursive-descent parser, a stateful interpreter and an access engine, and reading the two together shows the difference machinery-by-machinery. What they share is more striking: identity, diagnostics, a stable intermediate representation, targets derived from it, and tests based on invariants.

KEY INSIGHT

The best small compilers are not the ones with the most parts. They are the ones in which every part is present for a reason that can be stated, and every missing part is missing for a reason that can be stated too. That is what proportionality means, and this guide has tried to make both kinds of reason explicit.

◆ Where to go next

The appendices contain a complete reconstructed grammar with a tested corpus of lines, a glossary and an annotated reading list. A good next step is to write the failing tests suggested in chapters 6, 11 and 13 (trailing comments, the comma inside brackets, duplicate edges under different spellings), decide the policies they expose, and record each decision.

CHECK YOUR UNDERSTANDING

1. Explain to a colleague, in three sentences, why this compiler has no AST.
2. Choose one improvement from chapter 24 and outline its tests using the properties of chapter 21.
3. Which single idea from this guide would most improve a tool you maintain that reads text input?

Vikas Cohen

Compiler Architect and System Designer

This document describes the compiler-style architecture used by the ER DSL implementation as a conceptual and technical guide for transforming source declarations into semantic data, diagram geometry, and export artifacts.

Appendix A: A complete grammar and test derivation

This appendix collects the grammar and the recogniser in one place. The grammar is **reconstructed** from the informal grammar in chapter 8 and the details in chapter 6; where the original does not say, the entry is marked as an assumption.

◆ A. The grammar

```

program      = { line NEWLINE } [ line ]
line         = blank | comment | entity | relationship

blank        = { space }
comment      = "#" { any character }           (assumed: after leading spaces
are trimmed)

entity       = "entity" space { space } identifier
              [ { space } ":" { space } [ fieldList ] ]
fieldList    = field { "," field }
field        = { space } identifier [ "[" meta { "," meta } "]" ] { space }
meta         = { space } text-without-brackets-or-commas { space }

relationship = identifier "." identifier { space } operator { space } identifier { space }
}
operator     = "->" | "=>" | "~>"
identifier   = wordChar { wordChar }           (wordChar: a letter, a digit
or underscore)

```

The `field` and `meta` productions are the ones the informal grammar leaves out. They make explicit that commas appear at two depths: *between fields* and *between markers inside brackets*. A comma-splitter must treat them differently (chapter 6).

◆ B. The recogniser as implemented

PURPOSE	PATTERN (AS DESCRIBED IN THIS GUIDE)
Entity line	<code>^entity\s+(\w+)\s*(?:\s*(.))*?\$</code> with the case-insensitive flag
Relationship line	<code>^(\\w+)\\. (\\w+)\\s*(=> ~> ->)\\s*(\\w+)\\s*\$</code>

Two observations connect the patterns to the grammar: the entity pattern captures the field list as **free text** `(.)*`, so it is more permissive than the `fieldList` production, and both patterns are anchored, so trailing text after a relationship is rejected while trailing text after an entity's colon is absorbed.

◆ C. Behaviour of the two patterns on sample lines

Each line below was run through both patterns as written. The result is what those patterns do, and not a statement about what the compiler *should* do; the last column flags lines that suggest a policy decision.

LINE	RESULT	NOTE
<code>entity Account : Name, AnnualRevenue[Currency]</code>	entity	the chapter's example
<code>entity Account:Name</code>	entity	no space needed around the colon
<code>entity Account : Name</code>	entity	extra spaces are accepted
<code>entity Account :</code>	entity, empty field text	empty list
<code>entity Account</code>	entity, no field text	list is optional
<code>ENTITY Account : Name</code>	entity	keyword is case-insensitive
<code>entity 1Account</code>	entity	name starts with a digit: policy question
<code>entity Account__c : Name</code>	entity	custom-object style name
<code>entity Account : Name[Text(255)]</code>	entity	brackets and parentheses are free text
<code>entity Account : Name,</code>	entity, field text <code>Name,</code>	trailing comma: policy question
<code>entity Account : Name # primary</code>	entity, field text includes the comment	policy question
<code>entity Account Name</code>	no match	missing colon
<code>entity</code>	no match	no name
<code>entity Account</code>	no match	leading spaces: depends on a trim step
<code>Contact.AccountId -> Account</code>	relationship	the chapter's example
<code>Contact.AccountId->Account</code>	relationship	no spaces needed
<code>contact.accountid => account</code>	relationship	case is preserved by the capture
<code>Contact.AccountId ~> Account</code>	relationship	third operator
<code>Contact . AccountId -> Account</code>	no match	spaces around the dot
<code>Contact.AccountId --> Account</code>	no match	unknown operator
<code>Contact.AccountId -></code>	no match	no parent
<code>Contact.AccountId -> Account.Name</code>	no match	parent must be a bare identifier
<code>Contact.Account.Id -> Account</code>	no match	one dot only

LINE	RESULT	NOTE
<code>Contact.AccountId -> Account #</code> <code>note</code>	no match	anchored: policy question
<code>entity.Foo -> Bar</code>	relationship	child is named <code>entity</code>
<code>entity entity : Name</code>	entity	the entity is named <code>entity</code>

◆ D. From grammar to tests

A practical procedure for deriving tests from the grammar:

1. For every production, write the **smallest** valid string and a **largest reasonable** one.
2. For every terminal or constraint in a production, write a **near miss** that breaks only that constraint.
3. For every choice, write one string for each alternative and a string that could be mistaken for two of them.
4. For every optional part, write the string with and without it.
5. For every place where the recogniser is *more permissive than the grammar* (free text in the field list), write a test that records the current behaviour, and decide whether it is intended.

Appendix B: Glossary of terms

A working glossary of the terms used in this guide.

◆ Languages and recognition

TERM	MEANING
Alphabet	a finite set of symbols from which strings are built
Anchor	a regular-expression element (<code>^</code> , <code>\$</code>) that ties a match to the start or end of the text
Backtracking	trying one way to match and, on failure, returning to try another; the strategy of most regular-expression engines
BNF and EBNF	notations for writing grammars; EBNF adds repetition and optional parts
Chomsky hierarchy	the nested classes of formal languages: regular, context-free, context-sensitive and unrestricted
Context-free grammar	a grammar in which rules rewrite one symbol regardless of context; recognised by a pushdown automaton
Derivative (of a regular expression)	the expression describing what may follow a given prefix
Finite automaton	a machine with a fixed, finite number of states; recognises exactly the regular languages
Ordered choice	trying alternatives in order and taking the first that matches; the basis of parsing expression grammars
Prefix	the beginning of a string; completion works from a prefix
Pumping lemma	a result showing that regular languages cannot count without bound
Railroad diagram	a picture of a grammar rule as a path
Regular language	a language describable by a regular expression or a finite automaton
ReDoS	a denial of service caused by exponential backtracking in a regular expression
Scannerless parsing	recognising structure directly from characters, without a separate token stream
Stack	the unbounded memory that lets a machine recognise nesting

◆ Meaning and identity

TERM	MEANING
Canonical data model	a single agreed representation through which all formats are translated
Canonical form	a chosen representative of a set of equivalent things

TERM	MEANING
Closed-world assumption	what is not known to be true is taken to be false
Declarative	describing a result or structure rather than a sequence of actions
Denotational semantics	defining meaning by mapping constructs to mathematical objects
Equivalence relation	a relation that is reflexive, symmetric and transitive
Fold	combining a sequence into one value by applying a function step by step
Idempotent	applying an operation twice has the same effect as applying it once
Interning	ensuring there is exactly one object for each identity
Invariant	a property that must hold whenever the data is in a valid state
Metamorphic relation	a known relationship between the outputs for two related inputs
Open-world assumption	what is not known is simply unknown
Semilattice	a set with a join that is commutative, associative and idempotent
Symbol table	a data structure that maps names to what they denote

◆ Graphs and layout

TERM	MEANING
Connected component	a maximal group of vertices linked by paths
Feedback arc set	a set of edges whose removal makes a graph acyclic
Fan-in and fan-out	the number of edges arriving at, and leaving, a vertex
Induced subgraph	the vertices satisfying a condition and the edges among them
Lane	a parallel offset used to separate edges drawn between the same pair of boxes
Layered layout	a directed-graph drawing method with layers, crossing reduction and coordinate assignment
Level of detail	showing a summary instead of every detail when space is limited
Mental map	the user's memory of where things are in a diagram
Multigraph	a graph that may have several edges between the same pair of vertices, and self-loops
Reachability	whether one vertex can be reached from another by following edges
Self-loop	an edge from a vertex to itself
Topological order	an ordering in which every edge points from a later vertex to an earlier one
Transitive closure	the set of all pairs connected by a path

◆ Engineering

TERM	MEANING
Conservative extension	a change after which every existing program means what it meant before
Deforestation	fusing a producer and consumer so an intermediate structure is never built
Design by contract	specifying preconditions, postconditions and invariants for each function
Differential testing	comparing two implementations on the same input
Fitness function	an automated check that an architectural property still holds
Functional core, imperative shell	pure decision logic inside a thin layer that performs effects
Injective	mapping different inputs to different outputs
Memoisation	caching the result of a pure function
Output encoding	converting data so it is safe and correct in a target grammar
Pairwise testing	choosing tests so that every pair of parameter values appears together
Property-based testing	checking a stated law on many generated inputs
Provenance	a link from a derived artefact back to its origin
Tolerant reader	a reader that ignores or preserves what it does not understand

Appendix C: Further reading

Every entry below is a well-known title in its field. They are grouped by the question they help answer, with a note on how each connects to this guide.

◆ Compilers, parsing and formal languages

TITLE	WHY READ IT
<i>Crafting Interpreters</i> , Robert Nystrom (2021)	the friendliest hands-on path from scanning to a working language; freely available online
<i>Compilers: Principles, Techniques, and Tools</i> , Aho, Lam, Sethi and Ullman (2nd ed., 2006)	the classic reference for scanning, parsing, symbol tables and IR
<i>Introduction to the Theory of Computation</i> , Michael Sipser (3rd ed.)	regular languages, automata and the pumping lemma, clearly explained
<i>Introduction to Automata Theory, Languages, and Computation</i> , Hopcroft, Motwani and Ullman	thorough coverage of the Chomsky hierarchy
<i>Language Implementation Patterns</i> , Terence Parr (2010)	practical patterns for small languages
<i>Domain-Specific Languages</i> , Martin Fowler (2010)	the practical taxonomy of DSLs and when each pays off

◆ Regular expressions in practice

TITLE	WHY READ IT
"Regular Expression Matching Can Be Simple And Fast", Russ Cox (2007)	why backtracking engines can be exponential and automaton-based engines are not
Davis and others, "The Impact of Regular Expression Denial of Service (ReDoS) in Practice" (2018)	how common the problem is in real software
Thompson, "Regular Expression Search Algorithm" (1968)	the original construction of an automaton from a pattern
Brzozowski, "Derivatives of Regular Expressions" (1964)	the algebraic view used for completion
Ford, "Parsing Expression Grammars" (2004)	ordered choice as a foundation for recognisers

◆ Data models, graphs and layout

TITLE	WHY READ IT
Chen, "The Entity-Relationship Model: Toward a Unified View of Data" (1976)	the origin of the model this DSL describes
Codd, "A Relational Model of Data for Large Shared Data Banks" (1970)	the relational counterpart
Sugiyama, Tagawa and Toda, "Methods for Visual Understanding of Hierarchical System Structures" (1981)	the layered layout used by most diagram tools
Gansner, Koutsofios, North and Vo, "A Technique for Drawing Directed Graphs" (1993)	the layered layout in the <code>dot</code> tool
Di Battista, Eades, Tamassia and Tollis, <i>Graph Drawing</i> (1999)	a survey of graph-drawing algorithms
Misue, Eades, Lai and Sugiyama, "Layout Adjustment and the Mental Map" (1995)	keeping diagrams stable when they change
Tarjan, "Depth-First Search and Linear Graph Algorithms" (1972)	strongly connected components

◆ Architecture, design and testing

TITLE	WHY READ IT
<i>Enterprise Integration Patterns</i> , Hohpe and Woolf (2003)	the canonical data model pattern
<i>Object-Oriented Software Construction</i> , Bertrand Meyer	design by contract
<i>A Philosophy of Software Design</i> , John Ousterhout	deep modules and complexity
<i>Building Evolutionary Architectures</i> , Ford, Parsons and Kua	fitness functions
Parnas, "On the Criteria To Be Used in Decomposing Systems into Modules" (1972)	information hiding
Brooks, "No Silver Bullet" (1986)	essential and accidental complexity
Claessen and Hughes, "QuickCheck" (2000)	property-based testing
Wadler, "Deforestation" (1990) and "The Expression Problem" (1998)	two ideas used in chapters 7 and 23

◆ Tool documentation

RESOURCE	WHY READ IT
The Mermaid documentation for entity-relationship diagrams	the syntax and key markers used by the Mermaid backend
The draw.io (diagrams.net) file-format documentation	the XML structure the draw.io backend emits
The SVG specification	the format the canvas and PNG paths rely on

◆ A suggested study path

1. Read chapters 1 to 9 of this guide beside the first half of *Crafting Interpreters*.
2. Run the sample lines of Appendix A on your own build, and record each policy decision.
3. Read Cox's article on regular-expression matching to see why the patterns in chapter 6 are safe.
4. Skim Chen's paper and the layered-layout paper to see the origins of the model and the drawing.
5. Finish with Ford, Parsons and Kua to turn the invariants of chapter 25 into automated checks.